

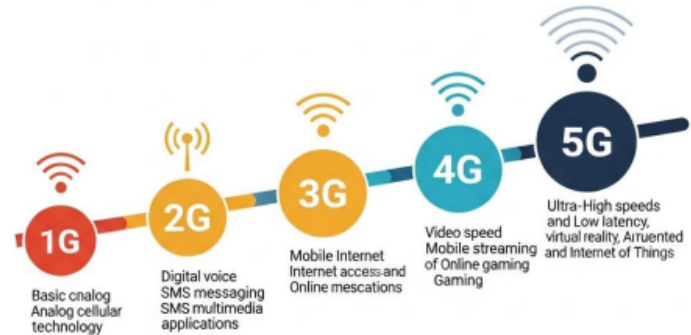
NETWORK APPLICATION

Martina Schirone - anno 2025/2026

The 6G Vision

1G - 5G

L'evoluzione delle reti mobili, documentata dagli anni '80 a oggi, rappresenta un percorso di trasformazione radicale: da semplici sistemi di comunicazione vocale si passa a piattaforme intelligenti e immersive.



Ogni generazione ("G") espande la precedente, introducendo cambiamenti architetturali e tecnologici fondamentali che migliorano l'esperienza degli utenti.

Linea temporale:

- 1980s: 1G - Analog Voice
- 1990s: 2G - Digital Voice & SMS
- 2000s: 3G - Mobile Data
- 2010s: 4G - Mobile Broadband
- 2020s: 5G - Diverse Services & IoT
- 2030s: 6G - Intelligent & Immersive Experiences

1G: L'era della mobilità analogica

La prima generazione si basava sulla tecnologia **FDMA** (Frequency Division Multiple Access), assegnando a ogni chiamata un canale radio dedicato (come nei walkie-talkie).

- **Caratteristiche:** Qualità della voce scarsa (statica), assenza di sicurezza (nessuna crittografia) e uso inefficiente dello spettro radio.
- **Impatto:** La vera rivoluzione fu la possibilità di comunicare senza vincoli fisici.

2G: La rivoluzione digitale e gli SMS

Qui si ha il passaggio ai segnali digitali e l'introduzione di tecniche di modulazione digitale come **TDMA** e **CDMA**.

- **TDMA (TIME DIVISION MULTIPLE ACCESS):**

Divide un'unica banda di frequenza in diversi time-slot in modo tale che essa sia condivisa tra più utenti. Ogni chiamata/sessione occupa un diverso time-slot e gli utenti si alternano.

- **CDMA (CODE DIVISION MULTIPLE ACCESS):**

A differenza del TDMA, non divide il canale per tempo, ma permette a più utenti di trasmettere contemporaneamente sull'intera banda di frequenza, distinguendo ogni segnale tramite un codice univoco assegnato a ciascun utente.

Vantaggi del 2G sul 1G:

L'introduzione di queste tecnologie rispetto alla precedente generazione (1G) ha portato a:

- **Efficienza e Capacità:** Un numero significativamente maggiore di utenti per ogni torre cellulare.
- **Qualità:** Una voce più chiara grazie alla correzione digitale degli errori.
- **Sicurezza:** L'introduzione della crittografia e dell'autenticazione tramite la SIM card.
- **Servizi Dati:** La nascita degli SMS (Short Message Service). Un servizio a banda bassa.

2.5G: Il passaggio ai Data

Prima del 3G, tecnologie di transizione dati come **GPRS** e **EDGE** hanno introdotto il concetto di connessione "sempre attiva" e la fatturazione basata sui megabyte anziché sui minuti.

- **GPRS (GENERAL PACKET RADIO SERVICE):**

Dati a commutazione di pacchetto: Rappresentava un'estensione (overlay) applicata alla rete 2G per permettere il trasferimento di dati a commutazione di pacchetto. (IP TRAFFIC)

Connessione "always-on": Ha introdotto per la prima volta il concetto di connessione "sempre attiva", eliminando la necessità di stabilire una connessione ogni volta che si volevano inviare o ricevere dati.

Nuovo modello di fatturazione: Con il GPRS, il costo del servizio è passato da una tariffazione "al minuto" a una basata sui megabyte di dati effettivamente scambiati.

Prime applicazioni internet: Ha abilitato servizi come la navigazione web tramite protocollo WAP e l'invio di messaggi multimediali (MMS).

Importanza storica: È stato il primo passo concreto verso l'Internet mobile, dimostrando l'enorme domanda del mercato per i dati in mobilità e aprendo la strada allo sviluppo del 3G.

- **EDGE (ENCHANTED DATA RATES FOR GSM EVOLUTION):** Una versione più veloce del GPRS.

3G: L'alba dell'Internet mobile

Sfrutta come tecnologia principale il **W-CDMA (Wideband CDMA)**, un'evoluzione della precedente tecnologia 2G CDMA che offre una banda di frequenza più grande, e **HSPA (High Speed Packet Access)**, ulteriore evoluzione che permette grandi velocità di scambio dati.

- **Architettura Basata Su Due Domini:** Mentre le generazioni precedenti (1G e 2G) erano nate quasi esclusivamente per la voce analogica o digitale, il 3G è stato progettato fin dall'inizio con l'obiettivo di offrire una larghezza di banda molto più elevata per supportare i dati.

L'architettura era però divisa in due domini:

- **Dominio a commutazione di pacchetto (Packet-Switched Domain)** per tutto il traffico dati IP
 - **Dominio a commutazione di circuito (Circuit-Switched Domain)** dedicato alla voce tradizionale
- **Impatto:** Ha reso il Mobile Internet una realtà. Col 3G si ha la navigazione web completa, chiamate video, email con file, navigazione GPS e la nascita degli App Store grazie ai potenziamenti HSPA.

4G/LTE: L'era della banda larga "All-IP"

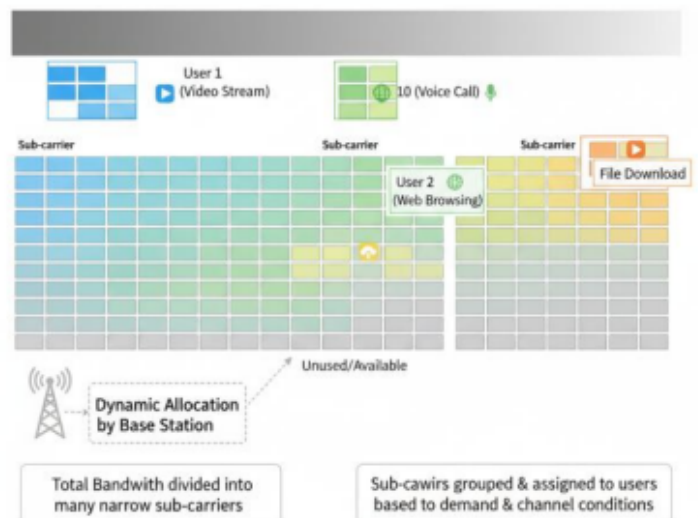
Le tecnologie radio utilizzate sono **OFDMA e MIMO**. Esse hanno permesso un salto di qualità dal 3G al 4G e aperto la via per il 5G.

OFDMA (Orthogonal Frequency-Division Multiple Access)

È la tecnologia utilizzata dal 4G per gestire l'accesso multiplo alla rete.

Come funziona: Divide l'intera larghezza di banda disponibile in moltissime piccole "sottoportanti" (sub-carriers) parallele. Queste vengono raggruppate e assegnate ai vari utenti in base alla loro domanda di dati e alle condizioni del canale.

Vantaggi: È estremamente efficiente dal punto di vista spettrale, resistente alle interferenze e permette un'allocazione delle risorse molto precisa, ideale per il traffico dati moderno.





MIMO (Multiple Input, Multiple Output)

Si riferisce all'uso di più antenne sia sul trasmettitore che sul ricevitore per inviare e ricevere più segnali di dati contemporaneamente.

Come funziona: Sfrutta il cosiddetto "multiplexing spaziale" per trasmettere flussi di dati diversi sulla stessa

frequenza. Nel **5G**, questa tecnologia si è evoluta in **Massive MIMO**, che utilizza array di centinaia di piccole antenne per servire molti utenti simultaneamente con grande precisione.

Vantaggi: Aumenta drasticamente la velocità di trasmissione dei dati (throughput) senza richiedere ulteriore spettro radio, migliorando sensibilmente la capacità della cella.

Architettura 4G:

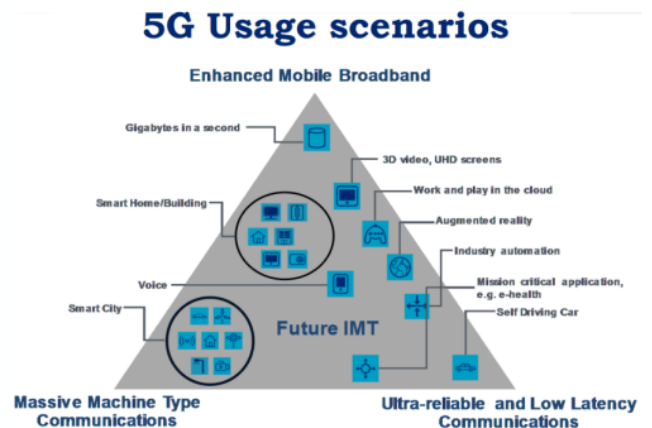
- **Architettura Unificata:** Col 4G si ottiene quello che il 3G non fu in grado di fare, ovvero ottenere un'unica architettura tutta basata su pacchetti a indirizzi IP. La commutazione di circuito dedicato alla voce viene infatti eliminata e la voce diventa una semplice applicazione dati **VoLTE (Voice over LTE)**.
- **Impatto:** Col 4G si ha una serie svariata di servizi tra cui:
 - **HD Video Streaming:** Youtube, Netflix
 - **Qualità delle Chiamate Video Alta:** FaceTime, Zoom, Teams.
 - **App Basate Sulla Geolocalizzazione:** Deliveroo, Uber.
 - **Boom dei Social Media:** Instagram, Snapchat, TikTok.

5G: Una Piattaforma Flessibile

Le generazioni precedenti si sono evolute considerando come unica metrica di paragone la qualità della voce e lo speed dei dati.

Il 5G invece è stato sviluppato per essere una piattaforma flessibile e unificata che si basa su **tre pilastri fondamentali**:

- **eMBB**
- **URLLC**
- **mMTC**



eMBB (Enhanced Mobile Broadband)

Focalizzato sull'offerta di una **banda massiva e velocità nell'ordine dei Gbps** (GigaBits per Second).

È il pilastro che abilita lo streaming video in 4K/8K, la realtà aumentata e l'accesso wireless fisso (FWA) ad alta velocità.

URLLC (Ultra -Reliable Low - Latency Communications)

Progettato per applicazioni critiche che richiedono una **latenza bassissima** (~1ms) e **un'affidabilità estrema** (99,999%).

È essenziale per la guida autonoma di veicoli, il controllo dei droni, la chirurgia remota e l'automazione industriale.

mMTC (Massive Machine -Type Communications)

Ottimizzato per **connettere un numero senza precedenti di dispositivi** (fino a 1 milione per km²). Si concentra su dispositivi a **basso consumo e bassa velocità di trasmissione**.

Fornisce la base per le Smart City (sensori per applicazioni come smart parking e luce intelligenti), Smart Agriculture (sensori del terreno) e l'IoT su larga scala.

Tecnologie Fondamentali Del 5G:

- **Nuovo Spettro:**
Utilizzo delle onde millimetriche (mmWave) per sbloccare larghezze di banda massicce.
- **Massive MIMO:**
Array di antenne con centinaia di elementi per servire più utenti simultaneamente tramite il beamforming.
- **Softwarizzazione (SDN/NFV):**
L'introduzione di Software-Defined Networking e Network Function Virtualization permette la virtualizzazione e softwarizzazione della rete e quindi una gestione agile ed efficiente delle risorse.

Architettura 5G:

Con 5G si passa da un'architettura prevalente hardware ad un'architettura software. Per fare questo passaggio ci si è basati sui seguenti principi:

- **SBA (Service-Based Architecture):**

La rete centrale non è più un insieme di apparati hardware rigidi, ma è ridisegnata come un insieme di **microservizi software** (Network Functions). Questi "comunicano" tra loro tramite **API standard** (proprio come le applicazioni web), rendendo la rete agile, facile da aggiornare e scalabile in tempo reale.

- **CUPS (Control and User Plane Separation):**

Separa logicamente il "**cervello**" della rete (Piano di Controllo, che gestisce le sessioni) dai "**muscoli**" (Piano Utente o UPF, che muove i dati). Questo permette di spostare i "muscoli" (l'inoltro dei dati) all'**estremità della rete (Edge)**, vicino all'utente, per eliminare i ritardi di trasmissione e supportare applicazioni ultra-rapide.

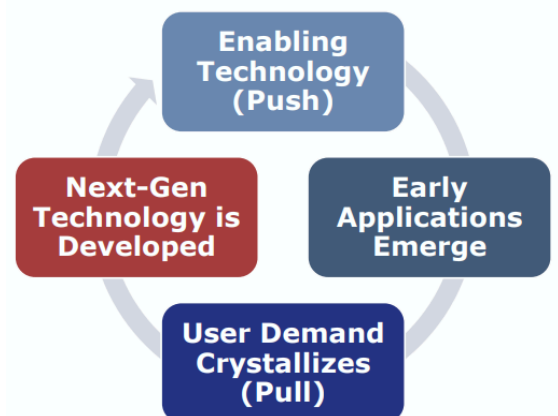
Limitazione del 5G:

Nonostante i progressi, il 5G presenta dei limiti che spingono verso il 6G:

- **Latenza End-to-End:** Sebbene l'interfaccia radio sia veloce (1ms), la comunicazione con cloud distanti richiede ancora 50-100ms, insufficienti per il "Tactile Internet".
- **Mancanza di Intelligenza Nativa:** Il 5G è altamente programmabile, esegue comandi ma non impara né si adatta autonomamente alle condizioni di rete.
- **Complessità di Scala:** Gestire migliaia di "network slice" per miliardi di dispositivi, tutti con requisiti diversi, richiede un'automazione superiore a quella nativa del 5G.
- **Gap di Sensing:** Il 5G è nato per comunicare, non per percepire l'ambiente fisico con precisione. Il 5G può connettere i sensori tra loro ma non fornisce funzioni di percezione dell'ambiente avanzate che tali sensori hanno bisogno per fare il loro lavoro.

Il Ciclo di Evoluzione:

- **Enabling Technology (Push):** Una nuova tecnologia viene introdotta.
- **Early Applications Emerge:** Applicazioni iniziano a sfruttare la nuova tecnologia per creare nuove funzionalità.
- **User Demand Crystallizes (Pull):** Gli utenti utilizzano le nuove app e la domanda per un servizio migliore aumenta.
- **Next-Gen Technology is Developed:** Viene sviluppata una nuova generazione che offre nuove tecnologie per soddisfare la domanda degli utenti.



La Visione del 6G: "Connected Intelligence"

Il 5G ha visto la nascita dell'intelligenza artificiale ma non ha avuto modo di essere sviluppata con essa, per questo la rete 5G rimane una rete flessibile, altamente programmabile ma non in grado di operare in maniera autonoma.

Il 5G non è in grado di percepire il mondo fisico che la circonda e adattarsi a questo.

Il 6G invece mira a fondere il mondo fisico, digitale e umano, trasformando la rete in un **sistema neurale distribuito**. Il 6G nasce con l'**intelligenza artificiale** come focus primario e ha tra i suoi obiettivi primari la **cura e sostenibilità dell'ambiente**.

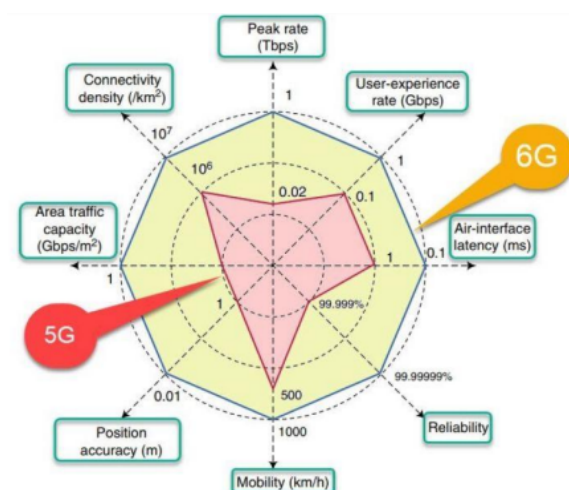
Obiettivi del 6G:

- **Esperienze Immersive:** Abiliterà la telepresenza olografica (avatar 3D fotorealistici) e il Metaverso persistente sincronizzato globalmente.
- **Tactile Internet:** Trasmissione del senso del tatto e del feedback aptico in tempo reale. **Sistemi Autonomi e Gemelli Digitali (Digital Twins):** Creazione di repliche digitali dinamiche di cuori umani, motori o intere città per simulazioni e diagnosi in tempo reale.
- **Sostenibilità:** Connessione globale tramite satelliti (NTN Non Terrestrial Network) ed efficienza energetica come obiettivo primario di progettazione ("Joule-per-bit").

Indicatori di Prestazione (KPI) del 6G

1. Velocità e Capacità dei Dati

- **Velocità di Picco (Peak Data Rate):** L'obiettivo è raggiungere circa **1 Terabit al secondo (Tbps)**, un incremento da 50 a 100 volte superiore rispetto al 5G.
- **Velocità Esperita dall'Utente:** Si punta a garantire stabilmente **1 Gigabit al secondo (Gbps)** quasi ovunque e in condizioni reali.
- **Perché sono necessari:** Queste velocità sono indispensabili per scaricare modelli olografici 3D ad alta fedeltà e per lo streaming di video volumetrici.



2. Reattività e Consistenza (Latenza e Jitter)

- **Latenza End-to-End (E2E):** L'obiettivo è scendere a **1 millisecondo (ms)** totali, rendendo le interazioni remote indistinguibili da quelle locali.
- **Latenza dell'Interfaccia Aerea:** Ci si riferisce alla latenza delle connessioni wireless. Si mira a livelli sub-millisecondo, arrivando potenzialmente all'ordine dei **microsecondi (μs)**.
- **Jitter (Variazione della latenza):** Deve essere **estremamente basso** per evitare problemi come la chinetosi (motion sickness) nelle applicazioni XR o la perdita di controllo nei sistemi tattili.
- **Perché sono necessari:** Avere latenze minime è necessario per avere reazioni istantanee e feedback aptico. Fondamentali per operazioni chirurgiche remote e realtà aumentata immersiva.

3. Affidabilità e Copertura

- **Affidabilità:** Il target è del 99,99999% ("**Seven Nines**"), che equivale a meno di 3,15 secondi di disservizio non pianificato all'anno.
- **Disponibilità:** La rete dovrà essere disponibile ovunque grazie all'integrazione di **Reti Non Terrestri (NTN)** come satelliti **LEO/MEO** e stazioni **HAPS**.
- **Perché sono necessari:** Per applicazione dove il fallimento non è un'opzione come operazioni chirurgiche remote o autopilot dei veicoli.

4. Densità e Posizionamento

- **Densità di Connessione:** Supporto per oltre **10 milioni di dispositivi per km²**, ovvero 10 volte la capacità del 5G (mMTC),
- **Precisione di Posizionamento:** **Accuratezza inferiore a 1 cm indoor e inferiore a 10 cm outdoor**, trasformando la rete in un sistema radar/GPS ad alta precisione.
- **Perché sono necessari:** Per abilitare l'Internet of Everything (IoE). Per creare una rete conscia del mondo fisico. Per la navigazione robot precisa.

5. Sostenibilità ed Efficienza

- **Efficienza Energetica:** Si punta a un miglioramento da **10 a 100 volte in termini di Joule-per-bit**, rendendo l'efficienza energetica un obiettivo primario.
- **Perché sono necessari:** Per ridurre i costi e l'impatto sull'ambiente.

6. Nuova Classe di KPI per il 6G

Oltre alle metriche convenzionali, il 6G introduce nuovi parametri per misurare le sue capacità intelligenti e di rilevamento:

- **Accuratezza del Sensing:** Misura la **precisione** con cui la rete **mappa l'ambiente fisico** (distanza, angolazione, composizione dei materiali).
- **Prestazioni dei Modelli di IA:** Valuta la **velocità di addestramento**, la **latenza** di processo dati e l'**accuratezza** dei modelli di intelligenza artificiale distribuiti nella rete.
- **Punteggio di Sicurezza/Fiducia (Security/Trust Score):** Una misura quantificabile in tempo reale della **resilienza** e della **sicurezza** della rete.

Tecnologie Fondamentali Del 6G:

Le tecnologie fondamentali che abiliteranno la visione del 6G includono:

- **Spettro Terahertz (THz):** L'uso di frequenze tra 0,1 e 10 THz per sbloccare una larghezza di banda enorme, necessaria a raggiungere velocità nell'ordine dei **Terabit al secondo (Tbps)**.
- **Superfici Intelligenti Riconfigurabili (RIS):** Superfici "specchio" controllate via software che possono riflettere o rifrangere i segnali per superare ostacoli fisici e ottimizzare la copertura. Risolve il problema della poca propagazione delle bande alte che sono facilmente bloccabili da ostacoli fisici come i muri.

- **Architettura AI-Native:** L'integrazione profonda dell'intelligenza artificiale in ogni livello della rete (non più come semplice aggiunta) per consentire l'auto-ottimizzazione e la gestione autonoma.
- **Sensing e Comunicazione Integrati (ISAC):** La capacità della rete di usare gli stessi segnali radio per trasmettere dati e contemporaneamente "percepire" l'ambiente circostante (come un radar), monitorando oggetti e movimenti.
- **Comunicazioni Semantiche:** Un cambio di paradigma che mira a trasmettere il **significato o l'intento** delle informazioni anziché ogni singolo bit, riducendo drasticamente l'uso di banda.
- **Sicurezza e Fiducia di Prossima Generazione:** L'adozione di **crittografia post-quantistica** per resistere a futuri attacchi informatici avanzati e l'uso di **Blockchain (DLT)** per la gestione trasparente di identità e transazioni.
- **Evoluzione del Cloud-to-Edge:** L'uso sinergico di **SDN** (programmabilità), **NFV** (flessibilità del software) e **MEC** per distribuire potenza di calcolo e intelligenza artificiale fino all'estremità della rete.

Generazione	Tecnologie Core e Fondamentali
1G (anni '80)	Radio Analogica, FDMA (Frequency Division Multiple Access).
2G (anni '90)	Modulazione Digitale, TDMA (utilizzato in GSM), CDMA .
2.5G / 2.75G	GPRS (General Packet Radio Service) per dati a pacchetto, EDGE .
3G (anni 2000)	W-CDMA (Wideband CDMA), HSPA (High-Speed Packet Access).

4G / LTE (2010)	All-IP Network (EPC), OFDMA, MIMO, VoLTE.
5G (2020)	SBA (Service-Based Architecture), CUPS, Massive MIMO, mmWave, SDN, NFV, Network Slicing.
6G (2030+)	Spettro Terahertz (THz), RIS (Superfici Intelligenti Riconfigurabili), AI-Native Architecture, ISAC (Sensing e Comunicazione Integrati), Comunicazioni Semantiche, NTN (Reti Non Terrestri).

Software-Defined Networking (SDN)

Nelle reti IP tradizionali, ogni dispositivo (router o switch) è un **sistema chiuso** e verticalmente integrato che comprende tre piani:

- **Data Plane:** L'hardware che si occupa di inviare i pacchetti dati.
- **Control Plane:** Il cervello che si occupa delle operazioni di routing.
- **Management Plane:** L'interfaccia di configurazione per gli operatori umani.

Data Plane

- **Funzione:** È focalizzato esclusivamente sull'elaborazione dei pacchetti a velocità di linea (line speed).
- **Compiti principali:** Consulta tabelle di inoltro pre-costruite (come la **FIB** - Forwarding Information Base) per determinare l'hop successivo, applica azioni (inoltro, scarto del pacchetto, modifica degli header, policy di QoS) e dirige il pacchetto alla porta di uscita appropriata.
- **Caratteristiche:** È ottimizzato per l'efficienza e la velocità, solitamente implementato tramite hardware specializzati come gli **ASIC**. Le prestazioni si misurano in pacchetti al secondo (PPS) o bit al secondo (BPS). (Application-Specific Integrated Circuits).

Control Plane

- **Funzione:** Determina lo stato della rete e **costruisce le tabelle di inoltro** utilizzate dal data plane usando i protocolli.
- **Protocolli:** Esegue protocolli di routing complessi (come **OSPF Open Shortest Path First** e **BGP Border Gateway Protocol**) per scambiare informazioni con i vicini e calcolare i percorsi ottimali, o protocolli di switching (come **STP Spanning Tree Protocol**) per prevenire loop.
- **Input/Output:** Riceve aggiornamenti di routing e cambiamenti della topologia dai dispositivi vicini; il suo output consiste nel popolare le tabelle FIB o MAC del data plane.
- **Caratteristiche:** In un'architettura tradizionale, la **logica di controllo è distribuita**: ogni dispositivo prende decisioni indipendenti. Viene solitamente eseguito su una CPU general-purpose all'interno del dispositivo.

Management Plane

- **Funzione:** Permette la configurazione, il monitoraggio e la risoluzione dei problemi (troubleshooting) del dispositivo.
- **Strumenti comuni:** L'interfaccia principale è la **CLI (Command-Line Interface)**, ma vengono utilizzati anche protocolli come **SNMP (Simple Network Management Protocol)** per il monitoraggio della salute del dispositivo o i più moderni **NETCONF/RESTCONF** per la telemetria e la configurazione automatizzata.
- **Limitazioni:** Nelle reti tradizionali, la gestione è di tipo **"box-by-box"**, il che significa che ogni dispositivo deve essere configurato individualmente. Questo rende l'applicazione di policy coerenti su larga scala un compito lento, complesso e soggetto a errori.

Limiti Delle Reti IP Tradizionali

- **Complessità Paralizzante:** La gestione avviene tipicamente **"box-by-box"**, ovvero ogni apparato deve essere configurato singolarmente tramite interfacce a riga di comando (CLI) specifiche del fornitore.

Questo approccio manuale è estremamente lento, costoso e soggetto a errori umani.

- **Gap di Agilità:** Le reti tradizionali non riescono a tenere il passo con la velocità delle applicazioni dinamiche e del cloud.

Mentre una macchina virtuale o applicazione può essere creata in pochi secondi, configurare la rete può richiedere giorni o settimane a causa dei processi di gestione manuali e statici.

- **Vendor Lock-In (Dipendenza dal Fornitore):** Il software del piano di controllo è proprietario e strettamente accoppiato all'hardware sottostante

Questo soffoca l'innovazione, poiché i gestori della rete non possono implementare nuovi algoritmi di rete personalizzati, ma devono attendere i rilasci software del fornitore.

Inoltre sono mantenuti elevati i costi di capitale (CapEx) e operativi (OpEx).

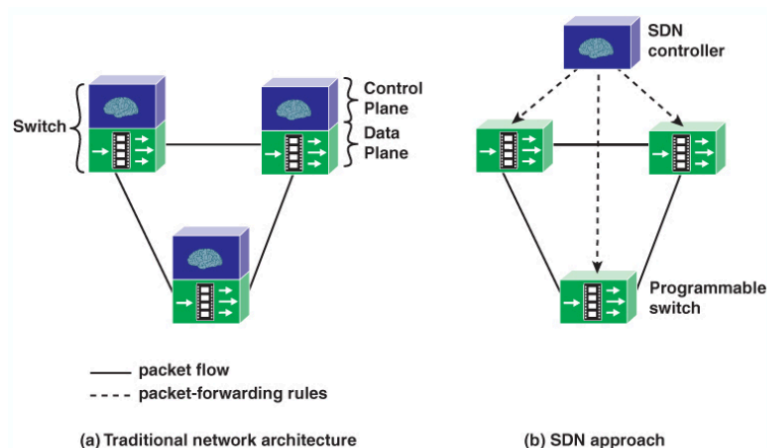
Il Concetto Cardine di SDN: Il Disaccoppiamento

L'idea centrale di SDN è la **separazione logica** tra il piano di controllo e il piano dati.

La logica di controllo (il "cervello") viene spostata dal dispositivo fisico a **un'entità software centralizzata** chiamata **SDN Controller**, mentre il data plane è **semplificato**, i dispositivi di rete diventano semplici elementi di inoltro programmabili.

Questo cambio di paradigma offre:

- **Visibilità Globale:** Il controller ha una visione in tempo reale dell'intera topologia, permettendo decisioni ottimali impossibili con protocolli locali.
- **Programmabilità e Automazione:** La rete può essere gestita tramite codice (Network as Code), automatizzando compiti ripetitivi.
- **Innovazione Aperta:** L'uso di interfacce standard permette di utilizzare hardware "white-box" economico con controller di diversi fornitori. Rompo il Vendor Lock-In.



Architettura SDN:

3 livelli:

- **Application Plane**
- **Control Plane**
- **Data Plane**

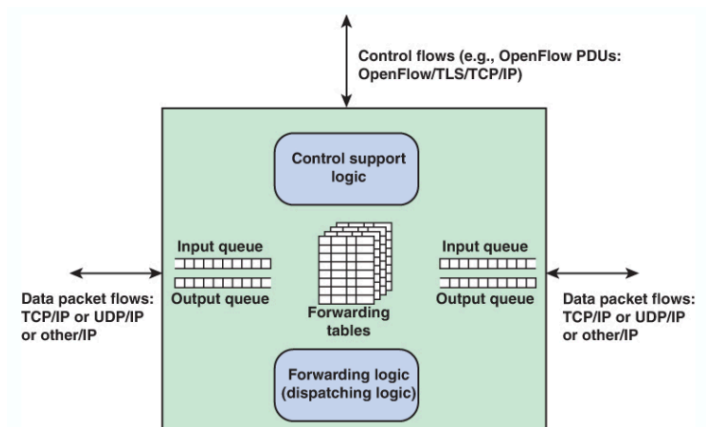
4 interfacce:

- Northbound & Southbound
- Eastbound & Westbound

Data Plane

Rappresenta i **"muscoli"** della rete e comprende sia i **dispositivi fisici** (switch, router) che quelli **virtuali** (vSwitch).

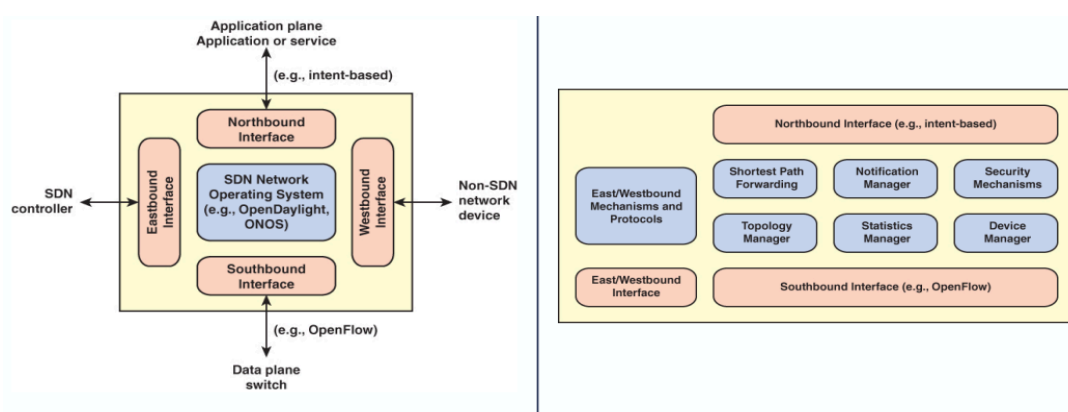
- **Ruolo:** Agisce come un semplice **elemento di inoltro programmabile**.
- **Funzionamento:** Esegue le istruzioni (regole di flusso) inviate dal piano di controllo. Non ha intelligenza propria per decidere i percorsi come nelle reti tradizionali.
- **Prestazioni:** È ottimizzato per il processamento dei pacchetti ad altissima velocità, operando direttamente nell'hardware specializzato come gli **ASIC**.
- **Feedback:** Segnala il proprio **stato** e le **statistiche sul traffico** al controller per permettere una gestione dinamica.



Control Plane

È il **"cervello"** della rete, implementato via software come un sistema operativo di rete (**Network Operating System/ SDN Controller**) eseguito su server standard. A differenza delle reti tradizionali, il controllo è logicamente **centralizzato**.

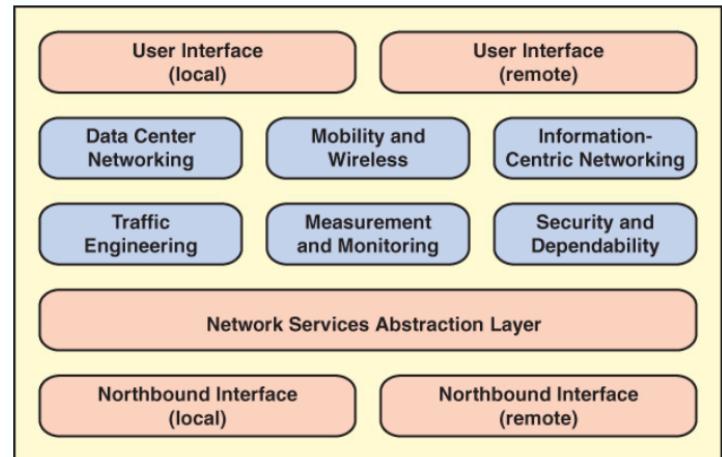
- **Visione Globale:** Il controller mantiene una **mappa completa** e in tempo reale dell'intera topologia. Con la quale calcola i percorsi ottimali per i dati tramite algoritmi di routing.
- **Traduzione:** Traduce le richieste ad alto livello provenienti dal piano applicativo in istruzioni di basso livello per i dispositivi del data plane.
- **Gestione Risorse:** Si occupa della **scoperta dei dispositivi**, della gestione delle risorse e dell'applicazione coerente delle policy in tutta la rete.



Application Plane

Questo strato contiene le applicazioni che implementano la **logica di business** e le **policy** di rete. Le applicazioni runnano sopra il SDN Controller.

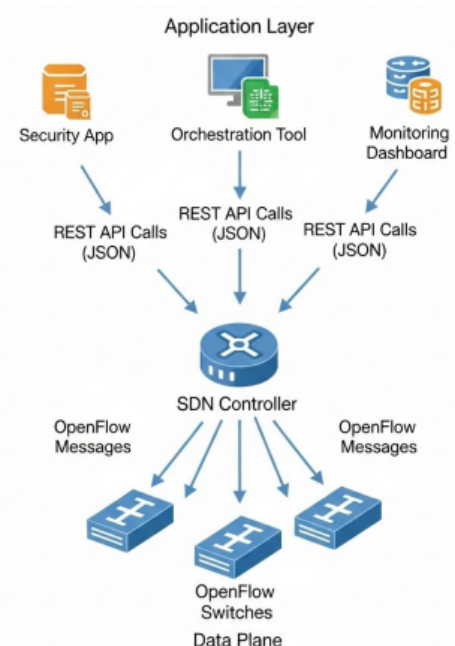
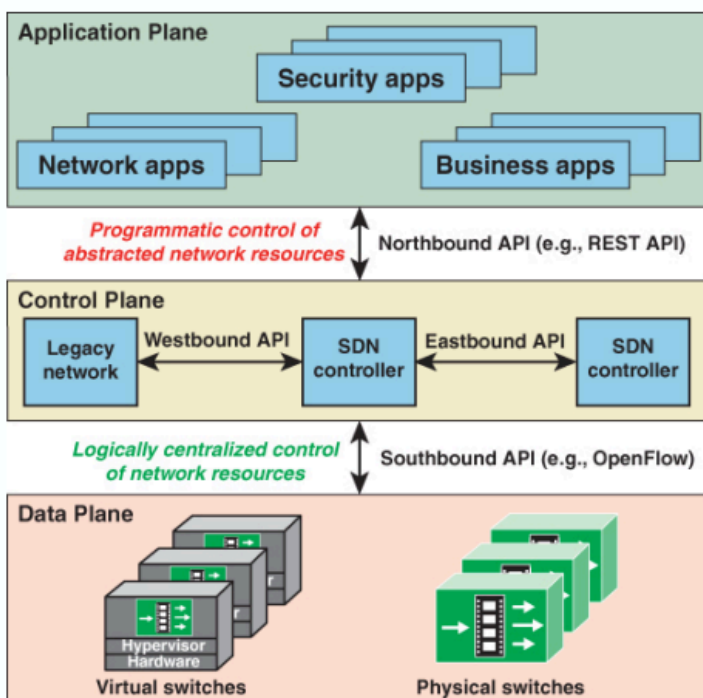
- **Funzionalità:** Sfrutta l'astrazione fornita dal controller per definire il comportamento della rete.
- **Esempi:** Include applicazioni per la sicurezza (firewall), il bilanciamento del carico, il monitoraggio o l'orchestrazione di servizi complessi.
- **Agilità:** Permette agli sviluppatori di **programmare la rete tramite API**, trattandola come una piattaforma software flessibile simile a un server o a un database.



Interfacce di Connessione

Il flusso di informazioni tra i piani è gestito da interfacce specifiche:

- **Northbound API:** Collega le applicazioni al controller; lo stile più diffuso è il **REST**, che utilizza metodi **HTTP** standard e **JSON** per definire l'intento della rete. Il suo obiettivo principale è quello di **nascondere la complessità** che c'è sotto.
- **Southbound API:** Permette al controller di programmare il data plane; il protocollo più noto e influente è **OpenFlow**.
- **Eastbound API:** Permettono la comunicazione tra diversi SDN controller in reti distribuite.
- **Westbound API:** Collegano il controller SDN a reti tradizionali (legacy).



Protocollo OpenFlow Delle SouthBound API

OpenFlow è un protocollo standardizzato che permette la comunicazione tra Data Plane (OpenFlow Switch) e Control Plane (SDN Controller). Permette al controller di programmare il data plane.

Modello OpenFlow Switch

- Ogni OpenFlow Switch presenta al suo interno una o più **tabelle di flusso**.
- Ogni tabella di flusso contiene al suo interno un set di **regole di flusso**.
- Una regola di flusso tipicamente consiste in 3 parti:
 - **Match Field:** Criteri per identificare i pacchetti in entrata (es. indirizzi IP, porte, ID VLAN). Usato come **filtro**.
 - **Istruzione:** Operazioni da eseguire in caso di match, come l'inoltro a una porta, lo scarto del pacchetto o la modifica degli header.
 - **Counters:** Statistiche sui pacchetti che hanno attivato quella specifica regola.

Pipeline di Matching

Quando un pacchetto arriva, viene confrontato con le voci delle tabelle in sequenza. Se non viene trovata alcuna corrispondenza (**Table Miss**), il pacchetto viene solitamente inviato al controller per ricevere istruzioni.

Messaggi Principali di OpenFlow

La comunicazione tra controller e switch avviene tramite diversi tipi di messaggi:

- **Dal Controller allo Switch:**
 - **Flow-Mod:** Il messaggio più comune, usato per **aggiungere, eliminare o modificare le regole** nelle tabelle di flusso; è lo strumento principale per programmare il comportamento della rete.
 - **Packet-Out:** Istruisce lo switch a inviare un pacchetto specifico da una determinata porta.
- **Dallo Switch al Controller:**
 - **Packet-In:** Inviato quando un pacchetto non trova corrispondenze nelle tabelle, permettendo al controller di decidere come gestire il nuovo flusso.
 - **Port-Status:** Notifica cambiamenti nello **stato delle porte** (es. link down), aiutando il controller a mantenere una visione aggiornata della topologia.
 - **Flow-Removed:** Informa il controller che una regola è stata rimossa, ad esempio per timeout.

Ciclo Vita di Un Pacchetto

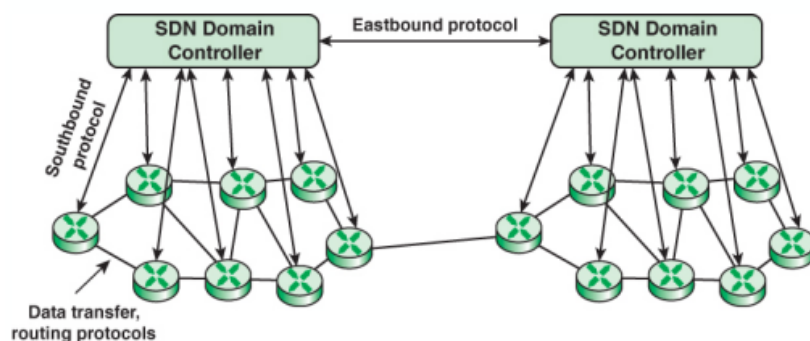
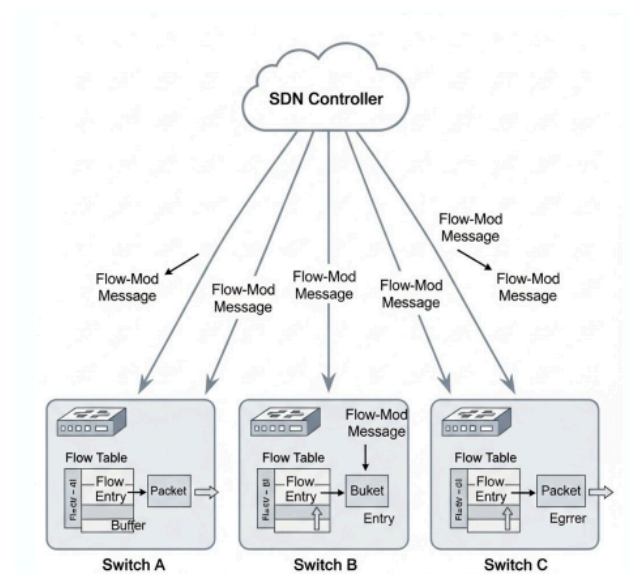
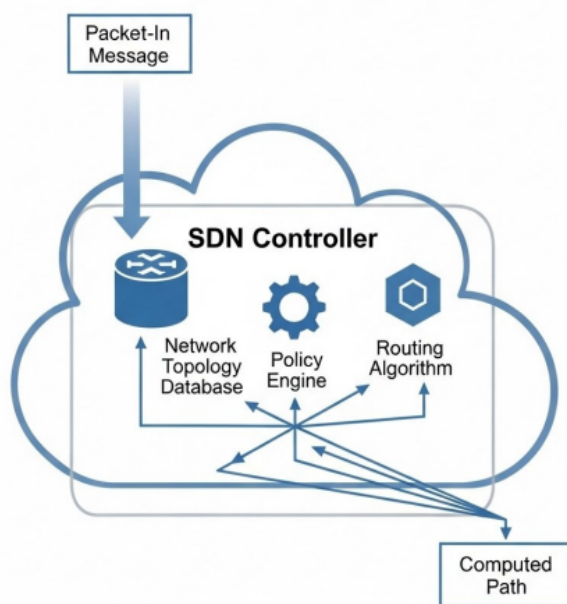
Quando arriva un pacchetto allo switch viene prima eseguita la **pipeline di matching**.

Se il pacchetto non corrisponde a nessuna regola esistente (**Table Miss**), lo switch lo invia al controller tramite un messaggio di **Packet-In**.

Nel mentre si aspetta una decisione del controller il pacchetto viene **salvato in un buffer** temporaneamente per evitarne la perdita.

Il controller analizza il pacchetto e il suo header, consulta le SDN applications, calcola il percorso ottimale in base alle policy globali e installa nuove regole negli switch lungo il percorso tramite messaggi di **Flow-Mod**.

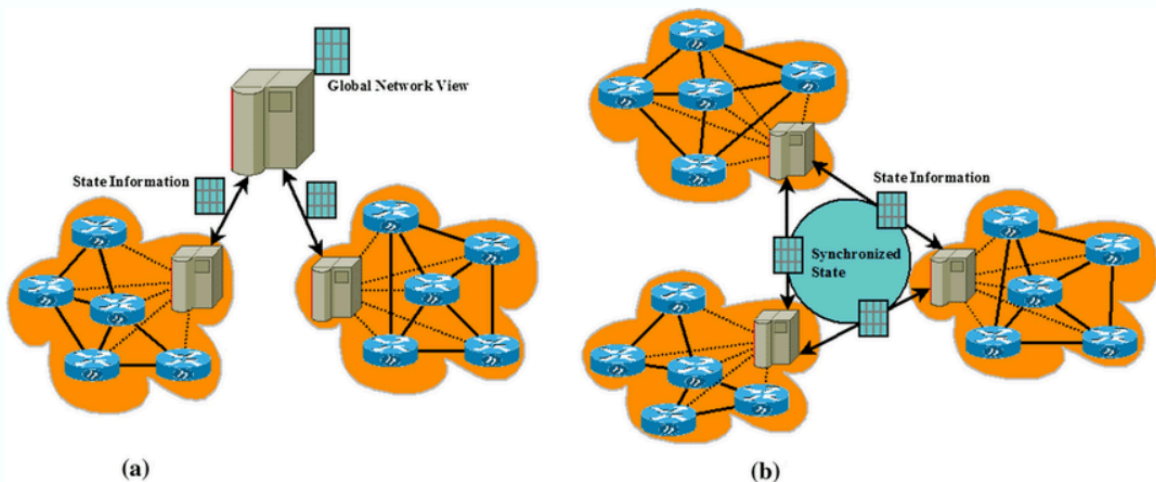
Tutti i pacchetti successivi dello stesso flusso vengono inoltrati direttamente dall'hardware a **line-rate** (massima velocità), senza ulteriori interazioni con il controller.



SDN Controller Design Patterns

Poiché il controller è il "cervello" del sistema, la sua progettazione è critica per la robustezza e la scalabilità della rete. Esistono tre modelli principali:

- **Modello Centralizzato:** Prevede un'unica istanza del controller che gestisce l'intera rete.
 - **Vantaggi:** Semplice da distribuire e garantisce nativamente una visione coerente della rete.
 - **Svantaggi:** Crea un singolo punto di guasto (Single Point of Failure) e può diventare un collo di bottiglia nelle prestazioni all'aumentare del traffico.
- **Modello Distribuito (es. ONOS, OpenDaylight)**
Il controllo è affidato a un cluster di istanze che lavorano in modo collaborativo.
 - **Vantaggi:** Alta disponibilità (se un'istanza fallisce, le altre subentrano) e scalabilità del carico di lavoro.
 - **Sfida:** La gestione dello stato, ovvero garantire che tutte le istanze abbiano una visione coerente e aggiornata della rete, risolta tramite protocolli di consenso come **Paxos** o **Raft**.
- **Modello Gerarchico**
Utilizzato per reti estremamente grandi o geograficamente disperse, implementa un "controller dei controller".
 - **Struttura:** Un controller di livello superiore (super controller) coordina diversi controller di dominio di livello inferiore.
 - **Vantaggi:** Scalabilità. Riduce la latenza delle risposte locali e permette ottimizzazioni specifiche per ogni dominio.
 - **Svantaggi:** Aumenta la complessità del design e richiede meccanismi di coordinamento robusti tra i livelli.



Reti Tradizionali VS SDN Controller

Caratteristica	Rete Tradizionale	SDN
Controller Plane	Distribuito su ogni dispositivo	Logicamente centralizzato nel Controller
Intelligenza del Dispositivo	Elevata (esegue protocolli complessi)	Bassa (semplice elemento di inoltro)
Configurazione	Manuale, dispositivo per dispositivo	Programmatica, a livello di rete, aperta
Vista della Rete	Frammentata, locale	Globale, centralizzata
Innovazione	Lenta, guidata dai fornitori	Rapida, guidata dal software

Network Function Virtualization (NFV)

Il Problema: "Middlebox Hell"

Prima dell'NFV, i provider e le aziende vivevano in quello che viene definito l'inferno delle middlebox, un modello dominato dall'hardware. Questo sistema prevedeva l'acquisto e l'installazione di numerosi apparati fisici proprietari e specializzati per ogni singola funzione di rete (firewall dedicati, router specifici, load balancer, ecc.).

- **Elevati costi iniziali (CapEx) e rischi:** L'acquisto di hardware specializzato richiede investimenti massicci e immediati. Esiste inoltre il rischio che l'hardware diventi **obsoleto** prima di aver completato il suo deprecation period.
- **Elevati costi operativi (OpEx):** Ogni scatola fisica occupa spazio, consuma energia e richiede sistemi di raffreddamento. Inoltre, richiede **personale specializzato** formato sui sistemi proprietari di ogni singolo fornitore.
- **Capacità bloccata (Stranded Capacity):** Poiché l'hardware viene acquistato per gestire i **carichi di picco**, rimane largamente inutilizzato durante i periodi di scarso traffico, rappresentando risorse costose che "siedono inerti".
- **Lentezza nel lancio di servizi (Time-to-Market):** L'implementazione di un nuovo servizio poteva richiedere dai **6 ai 12 mesi** a causa dei lunghi tempi di approvvigionamento dell'hardware, dell'installazione fisica e della configurazione manuale.

La Soluzione: Il paradigma NFV

Il principio fondamentale dell'NFV è che il valore risiede nel software (la logica funzionale) e non nell'hardware su cui gira.

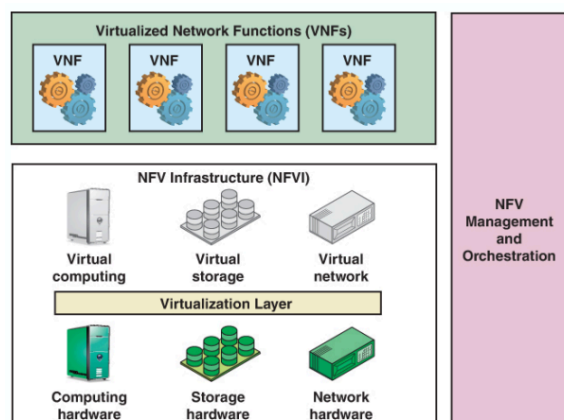
La soluzione proposta si articola in tre passaggi:

- **Disaccoppiamento:** Separare il software delle funzioni di rete dall'hardware proprietario.
- **Virtualizzazione:** Trasformare queste funzioni in **VNF (Virtualized Network Functions)** in grado di girare in ambienti virtuali.
- **Hardware Standard (COTS):** Eseguire le VNF su server e hardware standard, economici e ad alto volume (Commercial Off-The-Shelf), come quelli usati nei data center cloud.

L'Architettura ETSI NFV

Abbiamo 3 livelli:

- **Virtualized Network Functions (VNFs):**
Implementazioni software delle funzioni di rete (es. firewall, router).
- **NFV Infrastructure (NFVI):** Le risorse hardware (compute, storage, network) e il livello di virtualizzazione (hypervisor o container) che le astrae.
- **NFV Management and Orchestration (MANO):**
Il sistema che gestisce e alloca tutte le risorse dell'environment NFV.



NFV Infrastructure (NFVI)

È il livello che fornisce le risorse necessarie affinché il software di rete possa girare indipendentemente dall'hardware fisico
È composto da due elementi principali:

- **Risorse Hardware:** Include server standard ed economici (**COTS** - Commercial Off-The-Shelf), sistemi di **storage** (SAN/NAS) e switch/router fisici che garantiscono la connettività, il **networking**.
- **Livello di Virtualizzazione:** È il software che "estrae" l'hardware per creare risorse virtuali. Può basarsi su **Hypervisor** (come KVM o VMware) per creare macchine virtuali, oppure su **Container Engine** (come Docker) per soluzioni più leggere. Questo strato gestisce l'allocazione e l'isolamento di vCPU, vRAM e vSwitch per ogni singola funzione di rete.

L'NFVI può essere considerato un **cloud IaaS (Infrastructure as a Service)**, ma specificamente ottimizzato per il networking, dove sono richiesti alta velocità di elaborazione dei pacchetti e ritardi minimi.

Virtualized Network Functions (VNF)

Una VNF è l'**implementazione software** di una funzione di rete tradizionale (come firewall, router, load balancer o sistemi di analisi dei pacchetti).
È composta da due elementi principali:

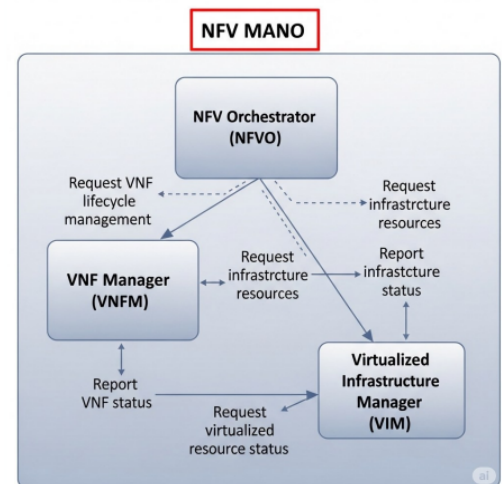
- **VNFC (VNF Components):** Una VNF è spesso composta da più componenti modulari chiamati VNFC, ciascuno dei quali svolge un compito specifico e gira solitamente all'interno di una propria macchina virtuale (VM) o container.

- **VNFD (VNF Descriptor):** È un file template (il "progetto" o "manuale d'istruzioni") in formato **YAML o XML** che contiene tutte le informazioni necessarie per il MANO.
 - **Requisiti di Risorsa:** Quantità di vCPU, vRAM e vDisk necessari.
 - **Immagini Software:** Posizione delle immagini (VM o container).
 - **Requisiti di Connettività:** Modalità di interconnessione interna tra VNFC e verso reti esterne.
 - **KPI (Key Performance Indicators) e Policy di Scaling:** Metriche di monitoraggio (es. utilizzo CPU > 80%) e regole per aggiungere o rimuovere istanze (Auto-scaling).

NFV Management and Orchestration (MANO)

Il sistema MANO automatizza l'intero ciclo di vita dell'ecosistema NFV attraverso tre entità gerarchiche:

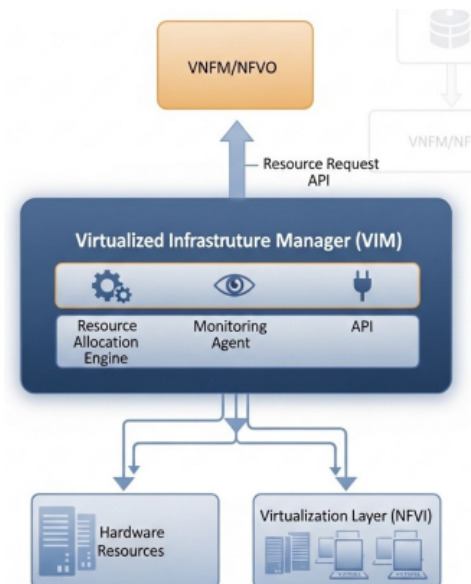
- **Virtualized Infrastructure Manager (VIM)**
- **VNF Manager (VNFM)**
- **NFV Orchestrator (NFVO)**



Virtualized Infrastructure Manager (VIM)

È il componente di livello più basso che interagisce direttamente con l'infrastruttura (NFVI).

- **Funzione:** Controlla e gestisce le risorse hardware (calcolo, storage, rete) e lo strato di virtualizzazione (hypervisor o container engine). Quindi le risorse del NFVI.
- **Compiti:** Si occupa di:
 - **Allocare e rilasciare** dinamicamente risorse virtuali (vCPU, vRAM, vDisk) su richiesta.
 - **Raccogliere dati sulle prestazioni** e sui guasti dell'hardware.
- **Interfaccia:** Espone API (Resource Request API) verso i componenti superiori.



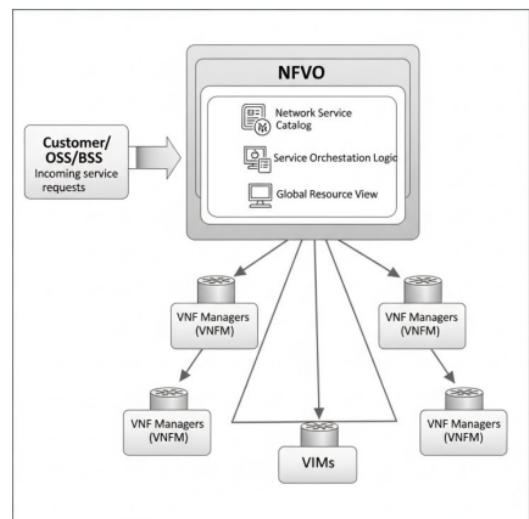
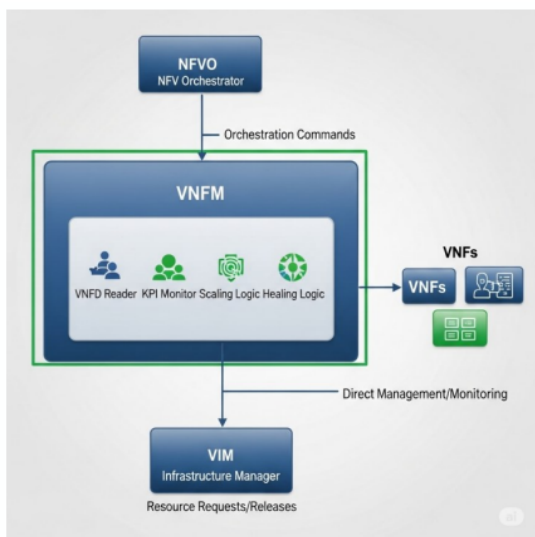
VNF Manager (VNFM)

- **Funzione:** Supervisiona l'intero ciclo di vita di una VNF.
- **Compiti:** Si occupa di:
 - **Installazione:** Crea un'istanza VNF consultando il VNFD (VNF Descriptor).
 - **Coordinazione col VIM:** Per l'allocazione di risorse hardware e virtuali.
 - **Scaling Up/Down:** Consulta i KPI e aggiunge o rimuove VNFC.
 - **Healing:** Individua guasti e attua il ripristino dei VNFC.
 - **Terminazione:** Termina il servizio VNF e rilascia le risorse.

NFV Orchestrator (NFVO)

Si trova al vertice della gerarchia MANO e fornisce l'intelligenza complessiva per gestire servizi complessi.

- **Funzione:** Orchestra i Network Services, che sono composti da **catene di più VNF** (ad esempio un firewall concatenato a un bilanciatore di carico).
- **Compiti:** Si occupa di:
 - **Onboarding:** Gestisce il catalogo dei servizi VNF
 - **End-to-End Orchestration:** Interagisce e coordina i vari VNFM e VIM per creare la catena di servizio e2e.
 - **Global Resource Management:** Mantiene una visione globale delle risorse disponibili tra diversi VIM o punti di presenza (NFVI-PoP).



Service Function Chaining (SFC)

L'SFC è il processo di creazione di un servizio di **rete end-to-end** indirizzando (steering) il traffico attraverso una **sequenza ordinata di funzioni di rete virtualizzate** (VNF). Invece di avere un percorso fisso attraverso apparati fisici, il cammino di un pacchetto è definito via software, indipendentemente dalla posizione fisica delle funzioni. Quello che fa il NFVO.

A differenza del modello tradizionale (dove cambiare un servizio richiedeva ri-cablaggi fisici o configurazioni manuali lente), l'SFC permette di:

- **Inserire, rimuovere o riordinare** le funzioni di rete (come firewall, NAT, load balancer) su richiesta.
- **Adattare la catena di servizi** in tempo reale in base alle esigenze delle applicazioni, alle policy degli utenti o alle condizioni della rete.

La Sinergia tra SDN e NFV

L'SFC rappresenta il punto d'incontro ideale tra queste due tecnologie:

- **NFV fornisce i blocchi costruttivi:** le singole VNF (i "muscoli" funzionali).
- **SDN fornisce l'idraulica ("plumbing"):** il controller SDN agisce come lo strumento che programma il tessuto di rete sottostante (specificamente i **vSwitch** nell'infrastruttura NFVI) per far fluire il traffico precisamente da una VNF alla successiva.

Le Limitazioni Del Paradigma SDN e NFV

- **La "Tassa" sulla Virtualizzazione:** L'esecuzione delle VNF su macchine virtuali introduce un overhead e latenza, causati principalmente dallo strato dell'hypervisor e dal virtual switch (vSwitch) che aggiungono process steps ulteriori rispetto al lavorare direttamente nell'hardware.
- **Colli di bottiglia dell'I/O:** Spostare i pacchetti tra la scheda di rete fisica (NIC Network Interface Card), l'hypervisor e la scheda di rete virtuale della VNF è un'operazione complessa e intensiva che i sistemi operativi tradizionali non sono progettati per gestire a velocità estreme.

Per superare questi limiti e raggiungere prestazioni simili al "bare-metal" (hardware fisico), è necessario utilizzare tecniche specializzate come **SR-IOV** per bypassare l'hypervisor e **DPDK** per bypassare il kernel del sistema operativo.

SR-IOV (Single Root I/O Virtualization)

È una tecnologia assistita dall'hardware che permette a una singola scheda di rete fisica (NIC) di **apparire come molteplici funzioni virtuali** indipendenti.

Consente alle funzioni di rete (VNF) di **accedere direttamente** alla scheda fisica **bypassando l'hypervisor e il virtual switch**, garantendo così una latenza minima e prestazioni quasi identiche a quelle dell'hardware fisico.

DPDK (Data Plane Development Kit)

È un insieme di **librerie software** che permette alle applicazioni di elaborare i pacchetti direttamente nello spazio utente, **bypassando il kernel** del sistema operativo.

Eliminando i rallentamenti tipici dello stack di rete tradizionale (come interrupt e passaggi tra kernel e app), il DPDK permette di gestire decine di milioni di pacchetti al secondo su server standard.

Cloud-Native Network Functions (CNFs) & Kubernetes

Le **Cloud-Native Network Functions (CNFs)** rappresentano l'evoluzione delle VNF basate su macchine virtuali, segnando il passaggio da applicazioni monolitiche a un'architettura basata su **microservizi containerizzati**.

Ogni micro servizio svolge una funzione specifica e indipendente (come l'autenticazione o la gestione delle sessioni) e comunica tramite **API RESTful standard**. I principi fondamentali delle CNF includono la **statelessness** (assenza di stato interno) per facilitare la scalabilità orizzontale, l'**immutabilità** dei container e l'integrazione di pipeline **CI/CD** per aggiornamenti rapidi e frequenti.

Kubernetes (K8s) si è affermato come lo standard di fatto per l'orchestrazione delle CNF, assumendo di fatto le responsabilità che nell'architettura NFV tradizionale appartenevano al **VIM** e al **VNFM**.

Kubernetes gestisce l'intero ciclo di vita dei container, solitamente organizzati in **Pod**, attraverso diverse funzioni critiche:

- **Deployment e Scaling:** Posiziona i container sui nodi disponibili e ne aumenta o diminuisce il numero automaticamente in base al carico di traffico.
- **Self-Healing:** Rileva i container guasti e li riavvia o sostituisce automaticamente per garantire la continuità del servizio.
- **Load Balancing:** Distribuisce il traffico in modo efficiente tra le varie istanze dei microservizi.
- **Gestione delle Risorse:** Alloca dinamicamente CPU, memoria e connettività di rete ai singoli container.

I principali vantaggi di questo approccio includono un'**agilità senza pari**, con tempi di distribuzione ridotti a pochi secondi, una maggiore **resilienza** ai guasti e un'**efficienza superiore** nell'uso delle risorse hardware rispetto alle macchine virtuali. Questa evoluzione è considerata fondamentale per abilitare scenari avanzati come il **Network Slicing** dinamico e i servizi di **Edge Computing** a bassissima latenza.

The Cloud-to-Edge Continuum

- **IaaS (Infrastructure as a Service):** Fornisce risorse virtualizzate di base (VM, storage, networking). È il modello principale per la NFV (Network Functions Virtualization), poiché gli operatori necessitano di un controllo granulare sullo stack di rete.
- **PaaS (Platform as a Service):** Offre una piattaforma completa per sviluppare e gestire applicazioni, nascondendo la complessità dell'infrastruttura sottostante.
- **SaaS (Software as a Service):** Consegna applicazioni pronte all'uso direttamente agli utenti finali tramite browser.

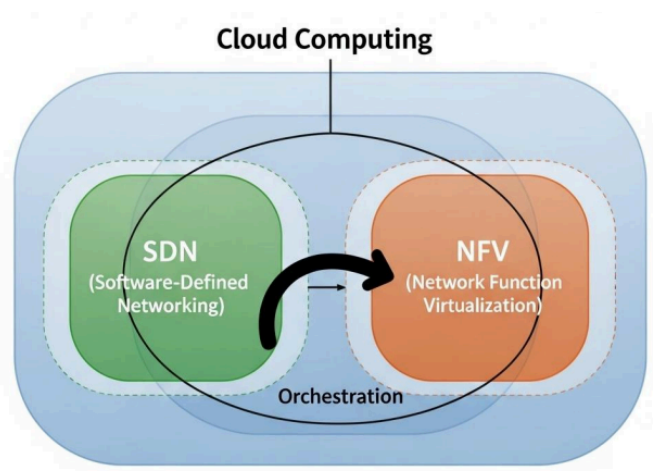
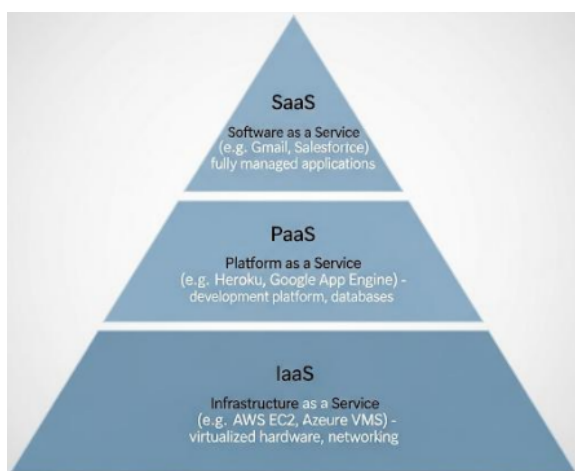
Simbiosi tra SDN, NFV e Cloud Computing

NFV, SDN e Cloud Computing rappresentano i **tre pilastri indipendenti** dell'infrastruttura moderna che, lavorando insieme, trasformano le reti da hardware statici in piattaforme agili e intelligenti, essenziali per il 6G.

In questa relazione simbiotica, ogni tecnologia svolge un ruolo specifico:

- **NFV (I blocchi costruttivi):** Disaccoppia le funzioni di rete dall'hardware proprietario, trasformandole in **VNF (Virtualized Network Functions)** che possono essere istanziate rapidamente via software e scalate modularmente.
- **SDN (Il controllo intelligente):** Agisce come il cervello del sistema, fornendo la **connettività programmabile** necessaria per collegare i blocchi virtuali (VNF) e sostituendo la configurazione manuale dell'hardware con policy software ad alto livello.
- **Cloud Computing (Il ponte fisico-virtuale):** Fornisce le risorse essenziali di calcolo, storage e rete richieste per eseguire sia le VNF che il controller SDN.

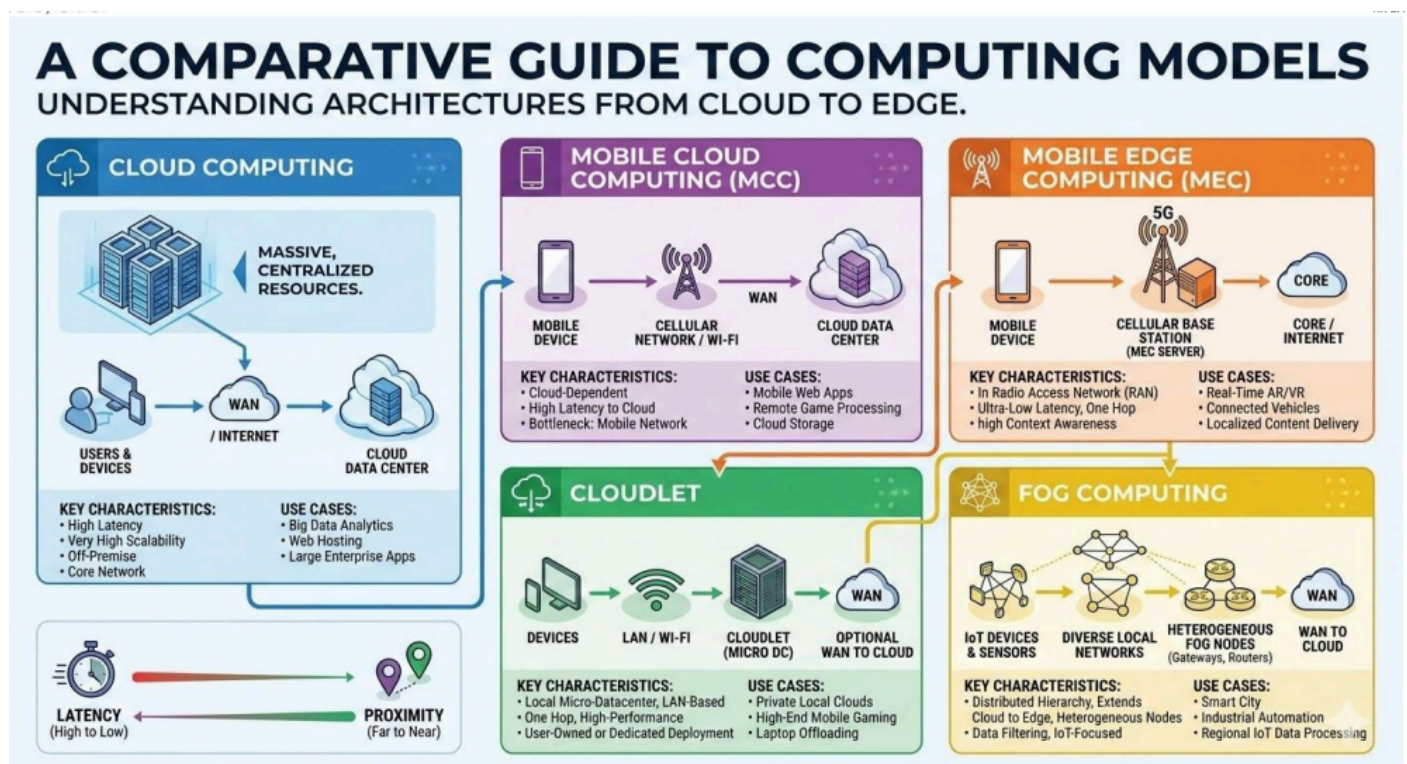
Questa sinergia abilita funzionalità avanzate come il **Service Chaining dinamico**, dove l'SDN dirige automaticamente il traffico attraverso una sequenza ordinata di VNF (es. firewall → DPI → load balancer) in base alle necessità, permettendo modifiche istantanee via software.



Il Limite del Cloud Centralizzato e la Soluzione Edge

Nonostante la potenza dei data center centralizzati, essi soffrono di un limite invalicabile: la **latenza** introdotta dalla distanza fisica.

- **Il problema della latenza:** Per applicazioni in tempo reale (auto a guida autonoma, chirurgia remota, AR/VR), ritardi nel tempo di andata e ritorno del segnale (**RTT**) verso cloud distanti è inaccettabile.
- **Edge Computing:** La soluzione consiste nello spostare l'elaborazione e lo storage **vicino** alla sorgente dei dati (all' "estremità" della rete). Questo riduce i tempi di risposta, risparmia larghezza di banda e migliora la sicurezza locale.



Lo Spettro del Cloud-to-Edge Continuum

Il **Cloud-Edge Continuum** rappresenta uno spettro continuo di potenza di calcolo e archiviazione dati che si estende dal dispositivo dell'utente fino ai più grandi data center centralizzati.

L'obiettivo fondamentale è distribuire l'intelligenza e i dati lungo questo spettro per ottenere l'equilibrio ottimale tra **velocità, costo e affidabilità**.

Lo spettro si articola in cinque livelli principali, ordinati in base alla vicinanza all'utente:

1. **Device/Sensor (Device Edge):** È il dispositivo fisico stesso (smartphone, visore AR, auto), dove i dati vengono generati inizialmente e dove può avvenire un primo processamento istantaneo.
2. **Local Gateway (On-Premise Edge):** Server locali o gateway situati nello stesso spazio fisico dei dispositivi (come una fabbrica o un edificio). Si occupa di aggregare i dati e fornire un'elaborazione a bassa latenza per i dispositivi nelle immediate vicinanze.
3. **Access Network (Network Edge / MEC):** Risorse di calcolo situate all'interno dell'infrastruttura di rete, ad esempio presso una torre cellulare 5G. Questo livello serve più gateway locali e utenti, offrendo una latenza leggermente superiore rispetto all'on-premise ma molto inferiore al cloud centrale.
4. **Regional Data Center:** Data center di dimensioni ridotte che servono specifiche aree geografiche o metropolitane, gestendo dati e servizi aggregati per le posizioni di Network Edge circostanti.
5. **Hyperscale Centralized Cloud:** I grandi data center distanti (come AWS, Azure o Google Cloud). Forniscono capacità di archiviazione immense e potenza per analisi complesse, machine learning e processi pesanti che non richiedono risposte in tempo reale.

Perché questo continuum è fondamentale?

- **Gestione della Latenza:** L'Edge è essenziale per applicazioni critiche come la chirurgia remota o i veicoli autonomi, dove i ritardi del cloud centrale sarebbero inaccettabili.
- **Efficienza della Banda:** Elaborare i dati vicino alla sorgente ("filtering at the source") permette di inviare al Cloud solo dati aggregati o anomalie, risparmiando fino al 99% della larghezza di banda.
- **Orchestrazione Ibrida:** Gli sviluppatori possono decidere in modo flessibile quali parti di un'applicazione far girare all'Edge (per le prestazioni) e quali nel Cloud (per ottimizzare i costi).

Quando usare l'Edge?

Non sempre conviene usare Edge al posto del Cloud. Qui in seguito c'è una tabella che spiega quando è meglio usare cosa.

Feature	Ideal for the Edge	Ideal for the Cloud
Latency	Very low (5–20 ms)	High (>100 ms)
Bandwidth	Bandwidth savings	Requires high bandwidth
Processing	Real-time, rapid	Intensive, non-real-time, Big Data Analytics
Context	Location-aware	Global and aggregated view
Example	Facial recognition for access	Training the facial recognition model

Sfide di Orchestrazione e Gestione Dati Dell'Edge

Gestire un'infrastruttura distribuita presenta sfide uniche rispetto al cloud centralizzato:

- **Scala Massiva:** Gestire decine di migliaia di piccoli siti edge invece di pochi grandi data center.
- **Vincoli di Risorse:** I siti edge hanno potenza e spazio limitati.
- **Varietà Dei Siti Edge:** Hardware e software nei siti edge può variare drasticamente quindi bisogna avere un modello edge flessibile.
- **Connessione a Intermittenza:** Alcuni siti edge possono avere una connessione nettamente peggiore di altri quindi si deve garantire che essi siano in grado di operare autonomamente anche quando disconnessi dalla rete.
- **Gestione Dati:** È fondamentale la **Località dei Dati**, elaborare vicino alla sorgente per minimizzare latenza e banda.
Sorge però il problema della **Consistenza**: garantire che i dati all'edge siano sincronizzati con il cloud.

Formula per L'E2E Latency

$$L_{total} = (2 \times L_{prop}) + (2 \times L_{net}) + L_{proc}$$

- **L_{prop} (Propagation Delay):** Il tempo che il segnale impiega per passare fisicamente nella fibra ottica.
- **L_{net} (Network Delay):** Il tempo speso all'interno degli switch/routers tra un hop e un'altro.
- **L_{proc} (Processing Delay):** Il tempo che la VNF spende per processare la richiesta dell'utente.

Tabella Di Confronto Tra I Vari Tipi Di Edge

Ecco una tabella riassuntiva che confronta i diversi modelli di calcolo (Cloud, Mobile Cloud, Mobile Edge, Cloudlet e Fog Computing) basata sulle informazioni contenute nelle fonti:

Aspetto	Cloud Computing	Mobile Cloud Computing (MCC)	Mobile Edge Computing (MEC)	Cloudlet	Fog Computing
Definizione	Fornisce servizi di calcolo su Internet.	Combinazione di cloud e calcolo mobile per app e servizi su dispositivi mobili.	Estende le capacità cloud all'estremità della rete, vicino alla sorgente dati.	Datacenter cloud su piccola scala situato ai margini di Internet.	Infrastruttura decentralizzata che pone storage e calcolo all'edge della rete.
Uso Primario	Calcolo general-purpose, storage e networking.	Potenziamento di applicazioni mobili tramite risorse cloud.	Riduzione della latenza e miglioramento performance per app mobili.	Supporto ad applicazioni mobili intensive in risorse con bassa latenza.	Supporto IoT e applicazioni real-time con bassa latenza e alta banda.
Latenza	Elevata (>100ms) a causa della distanza dagli utenti.	Moderata, dipendente dall'infrastruttura cloud.	Ultra-bassa (<10ms) e prevedibile grazie alla vicinanza.	Molto bassa, tipicamente a un solo hop wireless.	Bassa/Media, dovuta al processamento locale.
Scalabilità	Altissima, con risorse virtualmente illimitate.	Scalabile, sfruttando l'infrastruttura cloud.	Scalabile, ma limitata dalle capacità del nodo edge.	Scalabile all'interno dell'ambiente locale.	Scalabile, sfruttando sia risorse locali che cloud.
Luogo di Deployment	Centri dati centralizzati (Hyperscale).	Cloud datacenter con accesso mobile.	All'estremità della rete mobile (es. gNodeB/RAN).	All'estremità di Internet, vicino ai dispositivi mobili.	Vicino a dispositivi IoT e sorgenti dati (eterogeneo).

Esempi	AWS, Google Cloud, Microsoft Azure.	App mobili basate su cloud, MBaaS.	Reti 5G, AR/VR, analisi real-time.	Cloud gaming, assistenza cognitiva indossabile.	Smart grid, veicoli connessi, IoT industriale.
Vantaggi	Cost-effective, flessibile, accessibile ovunque.	Indipendente dalla piattaforma, analisi real-time.	Alta banda, processamento real-time, decisioni rapide.	Ridotta latenza, migliori prestazioni e durata batteria.	Risparmio di banda, QoS migliorata, analisi all'edge.
Sfide	Latenza, privacy dei dati, dipendenza dalla connettività .	Connettività fluida, sicurezza, vincoli energetici.	Complessità di deployment, sicurezza, integrazione infrastrutturale.	Complessità di implementazione, gestione delle risorse.	Sicurezza, privacy, interoperabilità tra dispositivi eterogenei.
Standard / Integrazione	Basato su economie di scala.	Dipendente dal cloud centrale.	Standardizzato da ETSI e 3GPP; accesso privilegiato alle info di rete.	Basato su LAN.	IEEE 1934; gerarchia distribuita cloud-to-edge.

Mobile Edge Computing (MEC)

Il tempo di andata e ritorno (RTT) di un pacchetto non è un valore unico, ma la somma di diversi ritardi lungo il percorso tra il dispositivo mobile (UE) e il cloud centrale.

I componenti del ritardo analizzati sono:

- **Processamento UE** (T_{ue}): ~1-2 ms.
- **Interfaccia Radio** (T_{air}): ~2-10 ms (variabile a causa della pianificazione radio e ritrasmissioni).
- **Processamento RAN** (T_{ran}): ~1-3 ms presso la stazione base (gNodeB).
- **Trasporto e Backhaul** ($T_{transport}$): ~5-20 ms per il collegamento dal sito cellulare alla rete network.
- **Processamento Network** (T_{core}): ~1-3 ms presso la User Plane Function (UPF) e i gateway.
- **Transito Internet** ($T_{internet}$): da 10 a oltre 150 ms per raggiungere il data center cloud.

L'impatto fondamentale del MEC consiste nel posizionare l'host di elaborazione direttamente presso la stazione base (RAN). In questo modo, il traffico viene interrotto localmente (local breakout), **eliminando completamente i ritardi di trasporto, core network e internet dal calcolo dell'RTT.**

Poiché queste sono le componenti più grandi e variabili del ritardo, il MEC permette di ottenere latenze ultra-basse e prevedibili, essenziali per le applicazioni in tempo reale.

Cos'è il MEC? La Definizione Formale

Il **MEC (Mobile Edge Computing)** è standardizzato dall'ETSI (European Telecommunications Standards Institute), che lo definisce come un sistema che fornisce un ambiente di **servizi IT** e capacità di **cloud computing** all'estremità della rete mobile, all'interno della **Radio Access Network (RAN)** e in stretta prossimità degli abbonati mobili.

Analizzando i termini della definizione:

- **Ambiente di servizi IT:** È progettato per far girare applicazioni standard con strumenti comuni, non funzioni proprietarie delle telecomunicazioni.
- **Capacità di Cloud Computing:** È elastico, virtualizzato, multi-tenant e programmabile tramite API.
- **Edge della rete mobile:** Si riferisce specificamente all'estremità gestita dall'operatore di rete.
- **All'interno della RAN:** È fisicamente co-locato con le infrastrutture di rete, come le stazioni base 4G/5G (gNodeB).

MEC vs. Fog: Una questione di ambito e integrazione

Sebbene entrambi i paradigmi portino il calcolo vicino all'utente, differiscono significativamente per portata e profondità di integrazione:

- **Fog Computing:** È un termine concettuale più ampio che descrive uno strato di calcolo distribuito ovunque tra il cloud e i dispositivi finali (può includere router aziendali, gateway industriali o risorse locali eterogenee).
- **MEC:** È un'implementazione **specificata e standardizzata** del Fog Computing, profondamente integrata nella rete mobile.

La **differenza chiave** risiede nel fatto che il MEC ha un **accesso privilegiato e standardizzato** alle funzioni e alle informazioni della rete mobile (come il carico della cella o la larghezza di banda in tempo reale) tramite API specifiche, cosa che un nodo Fog generico non possiede.

Feature	Centralized Cloud	Fog Computing	Mobile Edge Cloud (MEC)
Location	Centralized Hyperscale DC	Heterogeneous, opportunistic	Standardized, at the Mobile RAN
Latency	High (>30ms), variable	Low to Medium, variable	Ultra-low (<10ms), predictable
Network Context	None (over-the-top)	Limited (e.g., LAN info)	Rich (3GPP RAN info via APIs)
Mobility Support	Client-side only	Ad-hoc, non-standardized	Standardized (via 3GPP integration)
Key Standard		IEEE 1934	ETSI ISG MEC, 3GPP
Primary Enabler	Economies of Scale	IoT Gateways, local servers	5G Architecture (CUPS, UPF)

Perché usare MEC?

La Latenza

La Round-Trip Time (RTT) è la somma di vari ritardi. Il MEC elimina le componenti più ampie e variabili: il ritardo di trasporto, del processing network e del transito internet.

Non tutte le applicazioni internet hanno la stessa tolleranza ai ritardi:

- **Cloud-Friendly (>50ms):** Email, Big Data analytics, hosting di siti web.
- **MEC-Dependent (10-50ms):** Cloud Gaming, video analytics in tempo reale.
- **MEC-Essential (<10ms):** Internet tattile, Realtà Aumentata (AR), veicoli autonomi, robotica industriale.

Ottimizzazione della RAN (Radio Access Network)

La RAN è la parte più costosa e vincolata della rete. Il MEC permette di processare i dati localmente, evitando di intasare il backhaul verso il cloud centrale.

Esempio (Videosorveglianza):

- Una singola telecamera 4K genera ~15 Mbps.
- Una fabbrica con 100 telecamere genererebbe 1.5 Gbps di traffico uplink continuo.
- Con il MEC, l'elaborazione video avviene localmente e vengono inviati solo metadati o allarmi (es. "persona rilevata"), **riducendo la larghezza di banda** necessaria di oltre il 99%.

Contesto di Rete e Posizione

Le applicazioni MEC sono "**network-aware**". Attraverso API standardizzate, possono richiedere informazioni in tempo reale come:

- **Throughput** stimato per un utente specifico.
- **ID** della cella a cui è connesso un dispositivo.
- **Posizione** precisa del dispositivo all'interno della cella.

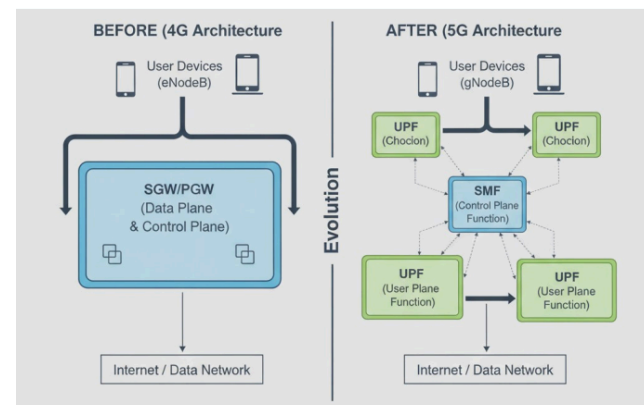
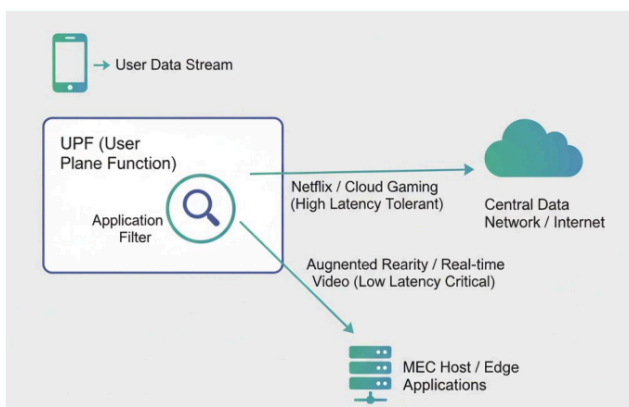
Privacy dei Dati

Il MEC consente di mantenere i dati sensibili (sanitari, aziendali o feed di sicurezza domestica) all'interno dei confini locali o nazionali, garantendo la conformità a leggi come il GDPR e prevenendo il transito dei dati sulla rete internet pubblica.

MEC nell'Ecosistema 5G

A differenza del 4G, il 5G è stato progettato con il MEC come principio nativo. I principali abilitatori 5G includono:

- **URLLC (Ultra-Reliable Low-Latency Communication):** Interfaccia 5G radio ad alte prestazioni.
- **NEF (Network Exposure Function):** Componente per esporre in sicurezza le capacità di rete alle applicazioni.
- **CUPS (Control and User Plane Separation):** La separazione tra il cervello della rete (Piano di Controllo, centralizzato) e il braccio operativo che instrada i pacchetti (Piano Utente UPF).
- **Local Traffic Breakout:** Meccanismo che consente alla UPF di intercettare il traffico (es. basato su un filtro per gaming AR a bassa latenza) e instradarlo direttamente all'host MEC locale, mentre il traffico normale (es. Netflix) prosegue verso la rete dati centrale.



Architettura MEC

2 livelli:

- **MEC Host Level:** Livello della singola entità fisica MEC e risorse.
- **MEC System Level:** Livello di gestione centralizzata e coordinazione di molteplici host.

MEC Host

Formato da 2 componenti:

- **MEC Platform:** L'ambiente software che fornisce servizi alle applicazioni.
- **NFV Infrastructure (NFVI):** Hardware (server x86 COTS) e strato di virtualizzazione.

Acceleratori Hardware del MEC Host

A causa dei vincoli fisici (spazio, potenza, raffreddamento) e delle prestazioni richieste, i MEC Host utilizzano spesso acceleratori specializzati:

- **GPU:** Per il rendering video (AR/VR) e l'inferenza AI parallela.
 - **PRO:** Facili da programmare
 - **CONS:** Consumano potenza e sono poco flessibili per task non parallele
- **FPGA (Field-Programmable Gate Array):** Circuiti hardware personalizzati per compiti di rete specifici ed efficienti.
 - **PRO:** Bassa latenza e consumo di energia.
 - **CONS:** Difficili da programmare.
- **Acceleratori AI (ASIC):** Progettati per l'inferenza di modelli AI ad alte prestazioni e basso consumo.

Virtualizzazione Del MEC Host

Le ME App possono essere considerate come **VNF specializzate** e progettate per interagire con i servizi della piattaforma MEC, per essere "consapevole" della rete radio o della posizione dell'utente.

- **Hypervisor-based (Virtual Machines):** Ogni ME App runna in una VM come si faceva nel NFV. Grande isolamento ma lente a partire.
- **Container-based:** Le ME App sono runnate in container. Approccio moderno dato che i container sono più leggeri e veloci delle VM.

Uso le stesse tecnologie di **SR-IOV (Single Root I/O Virtualization)** e **DPDK (Data Plane Development Kit)** per bypassare il vSwitch e kernel per ottenere performance migliori, proprio come facevo nelle NFV.

MEC Platform

Platform as a Service. Consiste in una collezione di servizi che sono esposti alle applicazioni tramite API.

Esempi di Servizi MEC:

- **RNIS (Radio Network Information Service):** Espone dati in tempo reale sulla qualità del canale radio e sul throughput.
- **TOF (Traffic Offload Function):** Regola il routing del traffico verso le app edge tramite regole basate su IP, tipo di app o profilo utente. Essenzialmente **filtra il traffico**.
- **Location Service:** Fornisce una localizzazione granulare fondendo dati GPS e misure radio della RAN.
- **Service Registry:** Un **registro** che tiene conto di tutte le ME App esistenti. Quando viene creata una nuova app si iscrive al registro dove espone il nome, porta e servizio.
- **Bandwidth Management Service:** Regola la **qualità di servizio** (banda minima, jitter, latenza e priorità di traffico) per garantire all'utente che l'app funzioni correttamente.

MEC Platform Manager

Cuore della MEC Platform. I suoi compiti sono:

- **Gestione dei Servizi di Piattaforma:** Supervisiona l'intero ciclo di vita di tutti i servizi offerti dalla piattaforma.
- **Sicurezza e Controllo Accessi:** Gestisce l'autenticazione e l'autorizzazione delle applicazioni, stabilendo quali app hanno il permesso di utilizzare specifici servizi.
- **Registro dei Servizi:** Mantiene aggiornato il Service Registry, che permette alle applicazioni di scoprire dinamicamente quali servizi sono disponibili e come comunicare con essi.
- **Interfaccia con l'Orchestratore:** Comunica con il MEC Orchestrator (MEAO) per coordinare la gestione del ciclo di vita delle Mobile Edge Applications (ME Apps).

Ciclo Di Vita Delle ME Apps

- **Onboarding:** L'applicazione è inserita nel catalogo dell'orchestratore.
- **Installation:** L'orchestratore seleziona un MEC Host in cui scaricare e far partire l'app.
- **Runtime:** L'applicazione sta running e può essere scalata up/down o migrata su un altro host.
- **Termination:** L'applicazione è fermata e le risorse sono rilasciate.

App State Management at the Edge

Cosa succede allo stato della sessione quando un utente passa da un MEC Host A ad un MEC Host B?

- **State Replication:** Copio lo stato della sessione all'host B per non perdere i dati.
- **A Tiered Architecture:** Conservo tutti i dati in un data center centralizzato a cui gli host possono accedere.

MEC System

Composto dal **MEC Orchestrator (MEAO)**.

MEC Orchestrator (MEAO)

Ha una **visione globale** dell'intero MEC System. I suoi compiti sono:

- **Catalogo ME App:** Mantiene il catalogo delle funzionanti ME App.
- **Ciclo Di Vita ME App:** Gestisce il ciclo di vita delle app insieme al MEC Platform Manager.
- **Inventario MEC Host:** Conosce tutti i MEC Host e le risorse in loro possesso.
- **Placement Delle Applicazioni:** Decide il migliore host su cui installare l'applicazione. Questo comporta problemi di ottimizzazione e off-loading.

$$\text{minimize } \sum (latency)_i \qquad \text{maximize } \sum (revenue)_i$$

- **Continuità Del Servizio:** Garantisce la continuità del servizio coordinando lo spostamento dei dati da un host ad un altro e l'installazione delle app sul nuovo host.

MEAO vs NFVO

Sono due concetti distinti ma spesso usati insieme.

- **NFV Orchestrator (NFVO):** È un orchestratore generico e **focalizzato sulle risorse**. Il suo compito è gestire qualsiasi tipo di funzione virtuale (VNF/CNF), assicurandosi che le risorse infrastrutturali siano allocate correttamente.
- **MEC Orchestrator (MEAO):** È un orchestratore specializzato e **focalizzato sul servizio**.

A differenza dell'NFVO, il MEAO è **consapevole della topologia** di rete; ad esempio, sa esattamente quale MEC Host è collegato a una specifica torre cellulare, informazione fondamentale per garantire le prestazioni ultra-basse richieste dalle applicazioni edge.

Le interfacce MEC

L'importanza delle interfacce standardizzate nell'architettura ETSI MEC risiede nella capacità di prevenire il **vendor lock-in**. Grazie a questi "punti di riferimento" comuni, gli operatori possono combinare componenti di fornitori diversi in un ecosistema aperto.

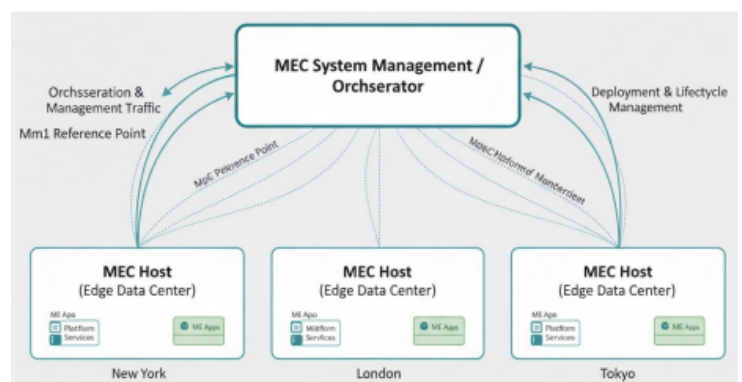
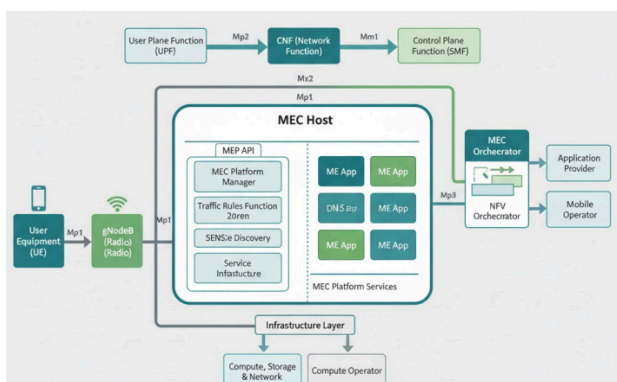
Interfacce del MEC Platform

- **Mp1 (Application-to-Platform):** È l'API RESTful che collega il **MEC Platform** alla **MEC App**. Attraverso questa interfaccia, le app scoprono i servizi disponibili e informazioni come la **localizzazione** e i dati radio (**RNIS**).
- **Mp2 (Platform-to-Infrastructure):** Collega il **MEC Platform Manager** al **VIM**. Viene utilizzata per richiedere la creazione di container o macchine virtuali e per configurare il networking virtuale necessario all'applicazione.
- **Mx2 (Platform-to-5G UPF):** È un'interfaccia critica per l'integrazione 5G. Collega il **MEC Platform** al **User Plane Function UPF**.

Permette alla piattaforma di inviare alla **UPF** le regole di filtraggio del traffico necessarie per attivare il **local breakout**, indirizzando i pacchetti dell'utente verso l'applicazione corretta.

Interfacce del MEC Orchestrator

- **Mm1:** Interfaccia tra il **MEC Orchestrator** (MEAO) e i sistemi di supporto operativo dell'operatore (**OSS**).
- **Mm3:** Collega il **MEC Orchestrator** al **MEC Platform Manager** di ogni singolo host; è essenziale per comandare l'installazione o la migrazione delle app.
- **Mm5:** Collega il **MEC Orchestrator** al **VIM**.



Computing And Off-Loading

Questo processo permette ai dispositivi mobili di superare i limiti di batteria e potenza inviando compiti pesanti all'edge.

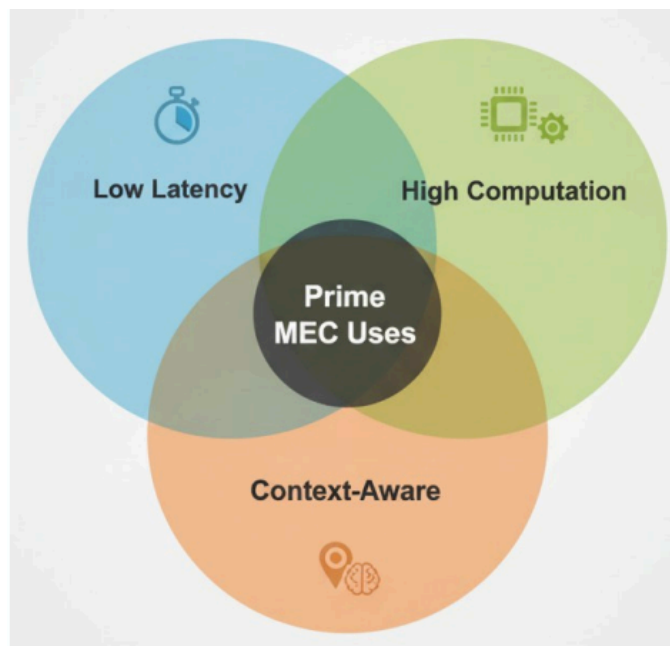
- **Il Trade-off Fondamentale:** Bisogna bilanciare il costo dell'esecuzione locale (alto consumo CPU e batteria) con il costo della comunicazione (tempo ed energia per inviare/ricevere dati).
 - **Total Latency (Local)** = $Time_to_compute_locally$
 - **Total Latency (Offload)** = $Time_to_send_input + Time_to_compute_on_MEC + Time_to_receive_output$
 - **Total Energy (Local)** = $Energy_to_compute_locally$
 - **Total Energy (Offload)** = $Energy_to_send_input + Energy_to_receive_output + Energy_consumed_while_idle$
- **Strategie:**
 - **Binary Offloading:** L'intero compito viene eseguito o localmente o sull'edge.
 - **Partial Offloading:** Il compito viene suddiviso in un grafo di sotto-task, decidendo quali parti esternalizzare.
- **Modello Dati:**
 - **Din (Input Data Size):** I dati da mandare al server per eseguire il task. Misurato in **bit**.
 - **C (Computational Complexity):** Il numero di CPU cycles per completare il task. Misurato in **Giga-cycles/cycles**.
 - **Dout(Output Data Size):** I dati da mandare al mobile device alla fine del task. Misurato in **bit**.
- **Modello Mobile Device:**
 - **flocal (Local CPU Speed):** Velocità di processo della CPU locale. Misurato in **cycles per seconds (Hz)**.
 - **Pcomp (Computation Power):** La potenza consumata dalla CPU locale. Misurato in **Watts**.
 - **Ptx (Transmit Power):** La potenza consumata dalla radio del device quando stiamo trasmettendo dati. Misurato in **Watts**.
 - **Prx (Receive Power):** La potenza consumata dalla radio quando stiamo ricevendo dati. Misurato in **Watts**.
 - **Pidle (Idle Power):** La potenza consumata dal device quando stiamo aspettando una risposta. Misurato in **Watts**
- **Modello Del Network:**
 - **Bup (Uplink Bandwidth):** Il data rate dal device al MEC host. Misurato in **bits per second**.
 - **Bdown (Downlink Bandwidth):** Il data rate dal MEC host al device. Misurato in **bits per second**.

- **Modello Del MEC Server:**

- **f_{mec} (MEC CPU Speed):** Velocità di processo della CPU del server MEC. Misurato in **cycles per seconds (Hz)**.
- **T_q (Queuing Delay):** Il server MEC è una risorsa condivisa che serve molti utenti. Se il server è occupato, il nostro task potrebbe dover attendere in una coda prima di essere elaborato.

- **Formule:**

- $Time_to_compute_locally = C / f_{local}$
- $Energy_to_compute_locally = P_{comp} \cdot Time_to_compute_locally$
- $Time_to_send_input = D_{in} / B_{up}$
- $Energy_to_send_input = P_{tx} \cdot Time_to_send_input$
- $Time_to_compute_on_MEC = C / f_{mec}$
- $Energy_consumed_while_idl = P_{idle} \cdot Time_to_compute_on_MEC$
- $Time_to_receive_output = D_{out} / B_{down}$
- $Energy_to_receive_output = P_{rx} \cdot Time_to_receive_output$



Network Slicing

Le generazioni precedenti (2G/3G/4G) utilizzavano un approccio "**one-size-fits-all**", ottimizzato esclusivamente per la banda larga mobile per utenti umani.

Questo crea conflitti con le moderne applicazioni che possono avere requisiti di rete completamente diversi l'una dall'altra: un'auto autonoma richiede latenza bassissima, mentre uno streaming video necessita di alta banda larga.

La soluzione a questo problema è stata trovata grazie al paradigma del **Network Slicing**, il quale permette di **dividere la rete in più fette**, ognuna delle quali soddisfa diversi requisiti di rete ed è quindi ottimizzata per diverse applicazioni.

Con il Network Slicing si ha il passaggio ad un modello **Network as a Service (NaaS)**. Ogni slice è un NaaS.

Definizione di Network Slicing

Una **Network Slice** è una **rete logica** che fornisce specifiche **capacità e caratteristiche di rete**.

- **Rete logica:** La slice è realizzata tramite **software e configurazione**, senza la necessità di nuovo hardware fisico dedicato.
- **Capacità specifiche:** Una slice può essere composta da un **set personalizzato di funzioni di rete** (ad esempio, una slice per l'IoT potrebbe non includere funzioni per i servizi vocali).
- **Caratteristiche specifiche:** La rete è configurata per offrire **livelli di performance garantiti**, come una latenza di 10ms o un'affidabilità del 99,999%.

I Quattro Pilastri Del Network Slicing

- **Natura End-to-End (E2E):** Include Radio Access Network RAN, Transport Network TN e Core Network CN.
- **Isolamento:** Le fette sono isolate in termini di risorse, prestazioni e sicurezza.
- **Personalizzazione:** Ogni fetta ha topologie e funzioni di rete su misura.
- **Automazione:** Creazione e gestione on-demand tramite API software

Tecnologie Del Network Slicing

L'implementazione del Network Slicing è resa possibile dalla convergenza di diverse tecnologie:

- **SDN (Software Defined Networking):** Controllo centralizzato e gestione dinamica delle risorse.
- **NFV (Network Function Virtualization):** Disaccoppiamento delle funzioni di rete dall'hardware, permettendo l'esecuzione di servizi come istanze virtualizzate su hardware COTS.
- **Cloud e Edge Computing:** Forniscono risorse computazionali scalabili e on-demand, elaborando i dati vicino alla sorgente per ridurre la latenza.

Network Slicing vs VPN vs QoS vs NFV

Aspetto	Network Slicing	Network Virtualization (NFV)	Quality of Service (QoS)	VPN (Virtual Private Network)
Definizione	Una rete logica end-to-end completa, autonoma e programmabile che include RAN, Trasporto e Core.	Una tecnologia abilitante che scollega le funzioni di rete dall'hardware proprietario per farle girare come software su server standard.	Un insieme di meccanismi per dare priorità ai pacchetti all'interno di un unico canale condiviso.	Una tecnologia che crea un tunnel criptato su una rete pubblica per garantire sicurezza e privacy.
Isolamento	Forte e multidimensionale: Un problema in una slice non influisce sulle altre.	Livello Infrastrutturale: isola le funzioni tramite hypervisor o container.	Inesistente: fornisce solo priorità relativa; un sovraccarico di traffico a bassa priorità può comunque degradare quello ad alta priorità.	Logico/Sicurezza: isola il traffico tramite crittografia, ma non garantisce l'isolamento delle performance dalle altre attività di rete.
Performance	Garantita	Dipende dall'ottimizzazione dello strato di virtualizzazione (es. DPDK, SR-IOV)	Relativa	Best-effort: la performance dipende interamente dalla rete sottostante

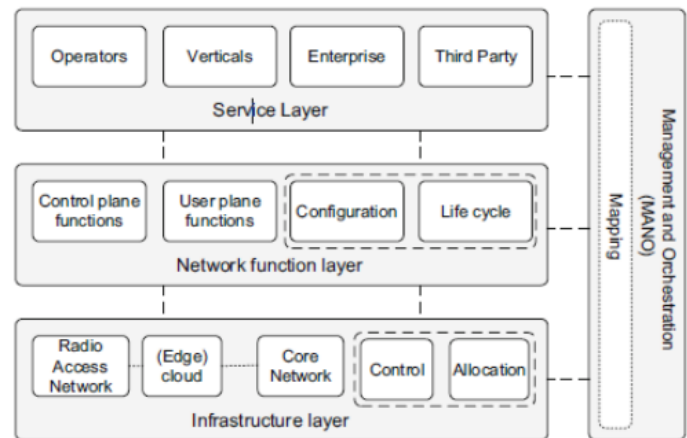
Sintesi delle differenze chiave:

- **NFV vs. Slicing:** NFV fornisce il **"cosa"** (le funzioni virtualizzate), mentre lo Slicing è il **"come"** (l'architettura che usa quelle funzioni per costruire una rete isolata end-to-end).
- **Slicing vs. QoS:** Il QoS è come una corsia preferenziale su un'autostrada trafficata (priorità relativa), mentre il Network Slicing è come costruire corsie fisicamente separate o tunnel paralleli dove il traffico altrui non può mai rallentarti (isolamento totale).
- **Slicing vs. VPN:** Una VPN si preoccupa della **privacy** (nascondere i dati), mentre una slice si preoccupa della **performance garantita** (assicurare che i dati arrivino in tempo e con la banda necessaria).

Architettura

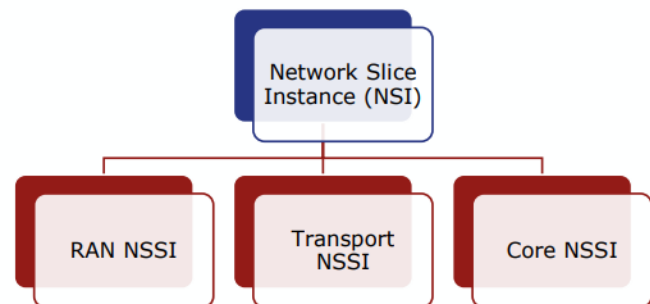
3 livelli e un orchestrator:

- **Service Layer:** Dove risiedono le applicazioni fornite da operator, imprese o terze parti.
- **Network Function Layer:** Gestisce la configurazione e il ciclo di vita delle funzioni di rete (User Plane e Control Plane).
- **Infrastructure Layer:** Include le risorse fisiche (RAN, Core, Edge Cloud) e le logiche di controllo e allocazione.
- **Management and Orchestration (MANO) Entity:** Coordina tutti i 3 livelli e regola il ciclo vita di una Network Slice.



Abbiamo poi la gerarchia architetturale:

- **Network Slice Instance (NSI):** La rete logica (slice) e2e completa esposta al cliente. Composta da un **set personalizzato di funzioni rete** e risorse allocate.
- **Network Slice Subnet Instance (NSSI):** Una porzione della fetta all'interno di un dominio specifico.
 - **RAN NSSI:** risorse radio.
 - **Core NSSI:** funzioni network.
 - **Transport NSSI:** risorse di banda.



Correlazione

- **Service Layer (o Business Layer) ↔ NSI (Network Slice Instance):** In questo livello superiore, il servizio richiesto dal cliente viene tradotto in una NSI.
- **Network Function Layer ↔ NSSI (Network Slice Subnet Instance):** Questo strato intermedio si occupa della configurazione e della gestione del ciclo di vita delle funzioni di rete.

Single Network Slice Selection Assistance Information (S-NSSAI)

Ogni fetta è identificata da un codice univoco composto da **SST (Slice/Service Type, 8-bit)** e un opzionale **SD (Slice Differentiator, 24-bit)** per distinguere, ad esempio, tra "eMBB per Enterprise" e "eMBB per Consumatori".

Tipologie di Slice (GSMA Generic Slice Templates)

La GSMA ha definito dei blueprint standardizzati denominati **Generic Slice Templates (GST)** per semplificare la creazione di servizi:

eMBB (Enhanced Mobile Broadband)

- **Ottimizzazione:** Per applicazioni con **alto data rate**, throughput in Gbps.
- **Casi d'uso:** Video Streaming 4K/8K, Fixed Wireless Access (FWA), AR/VR.
- **Implementazione:** Larghezze di banda ampie, medio bassa latenza.

URLLC (Ultra-Reliable Low-Latency Communications)

- **Ottimizzazione:** Latenza ultra-bassa (<5ms) e affidabilità estrema (>99.999%).
- **Casi d'uso:** Robotica industriale, veicoli autonomi (V2X), chirurgia remota.
- **Implementazione:** Meccanismi di scheduling mini-slot RAN, posizionamento dei server di calcolo all'estremo bordo (MEC).

mMTC (Massive Machine-Type Communications)

- **Ottimizzazione:** Densità di connessione massiva (>1 milione di dispositivi/km²), bassissimo consumo energetico da parte dei dispositivi connessi.
- **Casi d'uso:** Smart metering, tracciamento asset, agricoltura intelligente.
- **Implementazione:** Tecnologie Narrowband RAN (NB-IoT), funzioni network che possono gestire simultaneamente un grande numero di piccole sessioni.

L'Isolamento

L'**isolamento** è la caratteristica distintiva del **network slicing**.

Rappresenta la garanzia che le risorse, le prestazioni e la sicurezza di una slice siano protette dalle attività di tutte le altre che condividono la stessa infrastruttura fisica.

Senza un isolamento forte, una slice non sarebbe diversa da una semplice coda di priorità QoS (Quality of Service).

- **Isolamento Logico (Risorse Condivise):** Condivide le stesse funzioni network fisiche e virtuali. Efficiente ma meno robusto.
- **Isolamento Fisico (Risorse Dedicate):** Assegnazione di hardware specifico. Massima sicurezza, ma costo elevato e minore flessibilità.

Si possono distinguere tre tipi fondamentali di isolamento:

- **Isolamento delle Risorse:** Garantisce a ogni slice l'accesso esclusivo alla quota di risorse fisiche che le è stata assegnata. Questo avviene in ogni dominio della rete: nella **RAN** tramite blocchi di risorse radio dedicati, nel **Trasporto** tramite banda garantita e nel **Core/MEC** attraverso l'assegnazione di core CPU, RAM e storage dedicati nell'infrastruttura cloud.
- **Isolamento delle Prestazioni:** È la conseguenza diretta dell'isolamento delle risorse e assicura che le performance di una slice non siano degradate dalle altre. Questo risolve il problema del cosiddetto "**noisy neighbor**" (vicino rumoroso): ad esempio, un picco improvviso di traffico o un attacco DDoS sulla slice dedicata all'Internet pubblico non avrà alcun impatto sulla latenza o sulla banda garantita di una slice critica dedicata alla robotica industriale.
- **Isolamento di Sicurezza e Dati:** Assicura che il traffico delle diverse slice rimanga separato e che una violazione di sicurezza in una slice non possa compromettere le altre. Viene realizzato attraverso la **separazione del traffico** (tramite VLAN o VXLAN), l'uso di istanze di funzioni di rete dedicate (come un UPF separato) e la conservazione dei dati sensibili degli utenti in database isolati e controllati.

Slicing Della RAN

Lo **slicing della RAN (Radio Access Network)** è considerato uno degli aspetti più complessi del network slicing, poiché deve gestire risorse radio che sono per natura scarse, volatili e soggette a interferenze.

Allocazione delle risorse: i Resource Block (RB)

L'unità fondamentale delle risorse radio nel 5G è il **Resource Block (RB)**. Per isolare le diverse slice, la RAN adotta due strategie principali:

- **Partizionamento Statico:** Ogni slice ha una **quota fissa** di RB (es. 10% per URLLC). Garantisce un isolamento molto forte ma è inefficiente se una slice non ha traffico, poiché le risorse rimangono inutilizzate.
- **Partizionamento Dinamico:** Le risorse sono **condivise** ma ogni slice ha un **minimo** garantito e un **massimo** allocabile. È molto più efficiente grazie al guadagno di moltiplicazione statistica, ma offre un isolamento leggermente più debole.

Il RAN Scheduler "Slice-Aware"

Il componente software incaricato di decidere quale utente riceve quali RB in ogni istante è lo **Scheduler**. In una rete sliced, lo scheduler deve essere consapevole dei requisiti di ogni slice:

- **Per URLLC:** Usa algoritmi come *Earliest Deadline First (EDF)* per servire i pacchetti con scadenze urgenti, anche a costo di ridurre il throughput totale della cella.
- **Per eMBB:** Usa algoritmi come *Proportional Fair (PF)* per massimizzare il throughput totale quando la qualità del canale è ottimale.

Architettura Virtualizzata: CU/DU Split

Nelle reti 5G moderne, la stazione base (gNodeB) è disaggregata in tre parti:

- **Radio Unit RU:** antenna fisica.
- **Distributed Unit DU:** gestisce funzioni real-time.
- **Centralized Unit CU:** gestisce funzioni non real-time.

CU e DU sono implementate come funzioni software (VNF/CNF). È possibile istanziare CU e DU dedicate per ogni singola slice, garantendo un isolamento funzionale totale.

Modelli di Implementazione

L'ETSI e il 3GPP definiscono tre modelli principali basati sull'isolamento ed efficienza:

- **Modello A (RAN Condivisa):** CU e DU sono condivise; solo lo scheduler è consapevole delle slice. Massimo risparmio di risorse, minimo isolamento.
- **Modello B (CU Condivisa, DU Dedicata):** Isola il piano utente a livello di DU. Buon compromesso tra costi e performance.
- **Modello C (RAN Completamente Dedicata):** Ogni slice ha le sue istanze vCU e vDU. Massimo isolamento (sicurezza e performance) ma costo di risorse molto elevato.

Slicing Del Transport Network

Il **Transport Network Slicing** riguarda la gestione della "middle mile" della rete, ovvero la ciò che collega i siti RAN ai data center della Core Network. La sfida principale consiste nel fornire percorsi differenziati e larghezza di banda garantita per il traffico proveniente da diverse fette di rete.

L'architettura del trasporto si basa su due approcci principali:

- **Soft Slicing:** Fornisce una **separazione logica** su una rete a pacchetti condivisa utilizzando tunnel virtuali. È una soluzione **efficiente, flessibile e scalabile**, basata su tecnologie come **VPN** (VLAN, IPsec o MPLS) e tecniche di **Traffic Engineering** come il **Segment Routing (SRv6)** per indirizzare il traffico lungo percorsi ottimizzati. Tuttavia, offre garanzie statistiche piuttosto che deterministiche.
- **Hard Slicing:** Garantisce il massimo isolamento dedicando risorse fisiche o logiche fisse, come lunghezze d'onda in sistemi **DWDM** o slot temporali tramite **Flexible Ethernet (FlexE)**. Questo approccio crea dei "tubi rigidi" (hard pipes) che assicurano prestazioni deterministiche e isolamento totale, a scapito però della flessibilità e dell'efficienza delle risorse.

Architettura Moderna e Gestione

La gestione di questo dominio è affidata a due entità chiave:

- **Transport NSSMF:** Riceve i requisiti dall'orchestratore end-to-end e gestisce il ciclo di vita della sottorete di trasporto.
- **Transport SDN Controller:** Viene istruito dal NSSMF per calcolare il percorso ottimale e configurare i router (spesso tramite segmenti SRv6) per far rispettare le policy di latenza e banda richieste dallo SLA della slice.

Slicing Del Core Network

Il Core 5G è nativamente flessibile grazie alla **Service-Based Architecture (SBA)**, dove le funzioni di rete sono microservizi **stateless** che comunicano tramite API RESTful.

È quindi estremamente semplice istanziare, concatenare e scalare funzioni dedicate per ogni specifica fetta di rete.

Meccanismi di Implementazione

Esistono due approcci principali per isolare il Core Network:

- **Istanza Core Completamente Dedicata:** Un set completo di funzioni (AMF, SMF, UPF) viene distribuito esclusivamente per una slice. Offre il massimo isolamento ma è costoso in termini di risorse, ed è solitamente riservato a servizi critici come la pubblica sicurezza.
- **Modello Ibrido (Condivisione Selettiva):** Alcune funzioni comuni, come l'**AMF** (per la registrazione iniziale), vengono condivise tra più slice, mentre funzioni specifiche per il traffico dati, come l'**UPF**, vengono dedicate a ogni singola slice. Questo è il modello più comune poiché bilancia isolamento ed efficienza.

Isolamento del Piano di Controllo e dell'Utente

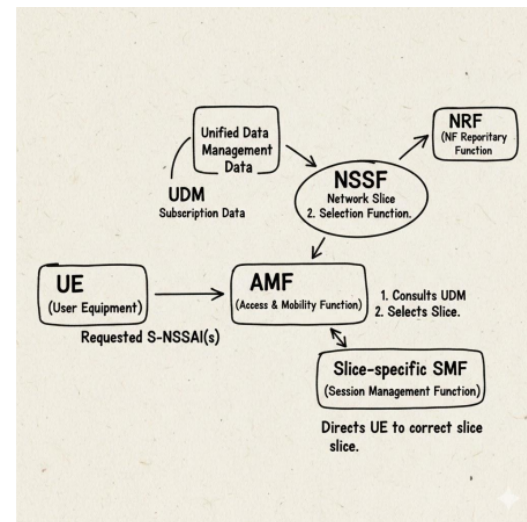
È fondamentale distinguere tra due tipi di protezione:

- **Isolamento del Piano di Controllo (CP):** Dedicare istanze di AMF o SMF assicura che una "tempesta" di segnalazione (ad esempio da milioni di dispositivi IoT) in una slice non faccia crashare il sistema per le altre.
- **Isolamento del Piano Utente (UP):** Un'istanza di **UPF dedicata** garantisce throughput e latenza deterministici, proteggendo la slice da attacchi DDoS o picchi di traffico video in altre zone della rete.

Procedura di Selezione della Slice

Quando un dispositivo (UE) si connette, avviene un processo coordinato tra varie funzioni core:

1. L'**UE** invia una richiesta contenente gli identificativi delle slice desiderate (**S-NSSAI**).
2. L'**AMF** riceve la richiesta e consulta il **NSSF** (Network Slice Selection Function).
3. Il NSSF verifica l'abbonamento dell'utente (tramite **UDM**) e consulta il **NRF**.
4. Infine, l'AMF indirizza la sessione dell'utente verso lo **SMF specifico** per quella slice, che configurerà l'UPF con le regole di QoS necessarie.



Gestione: Il Core NSSMF

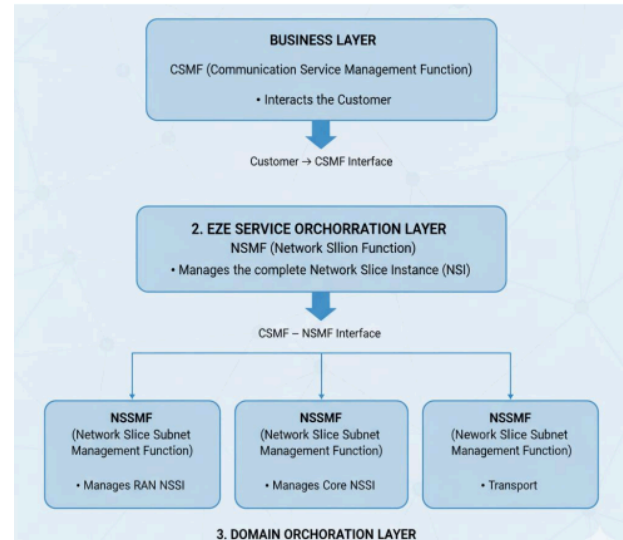
Il **Core NSSMF** è l'entità responsabile della gestione del ciclo di vita della sottorete core. Riceve istruzioni dall'orchestratore end-to-end (NSMF) e interagisce con il framework NFV (NFVO/VIM) per istanziare i container necessari, configurare le policy di rete e monitorare che i parametri di performance siano rispettati.

Slice Management & Orchestration MANO

La Gerarchia delle Funzioni di Gestione

L'architettura definita da 3GPP e GSMA si basa su tre funzioni logiche principali che operano in modo gerarchico:

- **CSMF (Communication Service Management Function):** È il portale rivolto al cliente (spesso **un'interfaccia web o API**) che traduce i requisiti di business (es. "voglio una slice per robotica industriale a Firenze") in ordini di servizio tecnico. Si occupa di validare la fattibilità commerciale e tecnica della richiesta.
- **NSMF (Network Slice Management Function):** Agisce come l'**orchestratore master**. Riceve l'ordine dalla CSMF e gestisce l'intero ciclo di vita della **Network Slice Instance (NSI)**. Il suo compito è scomporre i requisiti E2E in requisiti specifici per i singoli domini (RAN, Core, Trasporto) e coordinare i rispettivi gestori e risorse allocate.
- **NSSMF (Network Slice Subnet Management Function):** Gestiscono il ciclo di vita delle **Network Slice Subnet Instance (NSSI)**. Ricevono istruzioni dalla NSMF e le traducono in comandi di configurazione a basso livello per gli apparati e le funzioni di rete (VNF/CNF) del proprio ambito.
 - **RAN NSSMF:** Interagisce con il RAN Network Management System (NMS)
 - **Core NSSMF:** Interagisce con il NFV Orchestrator (NFVO) and VIM
 - **Teleport NSSMF:** Interagisce con il Transport Network's SDN Controller or NMS.



Service Level Agreement SLA

Contratto tra consumer e provider. Definisce in termini di **KPI** Key Performance Indicators la **performance garantita** dalla Network Slice. In breve definisce il comportamento che deve avere la slice.

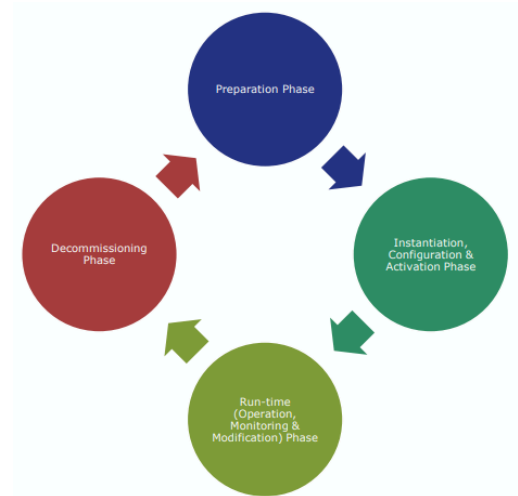
Network Slice Template NST

Una blueprint che contiene tutte le informazioni tecniche per fare il deploy di una specifica Network Slice. Contiene informazioni sulla configurazione della rete (VNF/CNF, topologia, funzioni network/microservizi...).

Il Ciclo di Vita di una Slice (Lifecycle)

L'orchestrazione gestisce la slice attraverso quattro fasi distinte:

1. **Preparation Phase:** Progettazione dei template (NST) e preparazione delle risorse prima dell'ordine del cliente.
2. **Instantiation, Configuration & Activation:** Creazione della NSI, allocazione delle risorse fisiche/virtuali e attivazione del servizio in base all'ordine.
3. **Run-time Phase:** Monitoraggio continuo delle prestazioni rispetto allo **SLA (Service Level Agreement)** e modifiche dinamiche come lo scaling.
4. **Decommissioning Phase:** Disattivazione della slice e rilascio delle risorse nel pool condiviso quando non sono più necessarie.

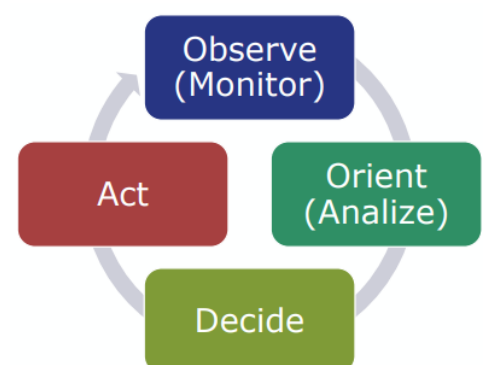


Slice Installation Workflow

Il flusso di lavoro si articola nelle seguenti fasi principali:

1. **Preparazione e Ordine del Servizio:** L'operatore prepara i template tecnici (NST). Il cliente, tramite un portale web (**CSMF**), ordina la slice definendo parametri come l'area geografica, il numero di dispositivi e lo **SLA** richiesto (es. latenza < 5ms).
2. **Inizio dell'Orchestrazione E2E:** La funzione **NSMF** riceve l'ordine, seleziona il template appropriato dal catalogo e agisce come "master orchestrator" coordinando i gestori dei singoli domini tecnologici (**NSSMF**).
3. **Creazione dei Subnet di Dominio:**
 - **Core:** Il Core NSSMF interagisce con il framework NFV per distribuire e configurare le funzioni di rete necessarie (AMF, SMF, UPF).
 - **RAN:** Il RAN NSSMF alloca le risorse radio (Resource Blocks) e configura lo scheduler per garantire le prestazioni richieste alla slice.
 - **Trasporto:** Il Transport NSSMF crea tunnel logici (es. tramite **Segment Routing**) e riserva la banda necessaria tra i siti RAN e il Core.

4. **Attivazione e Monitoraggio:** Una volta che tutti i domini confermano il successo, il NSMF attiva la **Network Slice Instance (NSI)**. Inizia immediatamente il processo di **Closed-Loop Assurance (OODA)**, dove il sistema monitora costantemente i KPI e interviene autonomamente in caso di potenziali violazioni dello SLA.



Esempio Di Allocazione Risorse

In questo esempio vengono mostrate come la Network Slice e le risorse vengono allocate/configurare per garantire i minimi KPI espressi nel SLA per reti di diversa tipologia.

Slice Type	RAN Allocation (100 PRBs)	Transport Allocation (100 Gbps)	Core UPF (200 Gbps)	Core Control Plane (500 vCPUs)
URLLC	10 PRBs (Static)	2 Gbps (Strict Priority)	1 Gbps (Prio 1)	20 vCPUs (Dedicated)
eMBB	10-70 PRBs (Dynamic)	45 Gbps (Guaranteed)	40 Gbps (Prio 2)	40 vCPUs (Dynamic Pool)
mMTC	5 PRBs (Static NB)	1 Gbps (Low Priority)	0.5 Gbps (Prio 5)	100 vCPUs (Dedicated Cluster)

Multimedia Networking

Il traffico di dati multimediali è molto più sensibile rispetto ad altri tipi di traffici. Ci sono determinate challenge e caratteristiche che lo distinguono.

Key Challenges

Larghezza di banda elevata: Video compressi e non compressi richiedono in ogni caso enormi bande larghe.

Sensibilità alla Latenza: Per applicazioni real-time (VoIP, video conferencing), il ritardo end-to-end deve essere basso (< 150-200 ms).

Sensibilità al Jitter: Jitter è la variazione del tempo di arrivo dei pacchetti. Un jitter alto causa dei video a scatti, i pacchetti devono avere un arrivo costante.

Tolleranza alle Perdite: Multimedia streams possono sopportare la perdita di qualche pacchetto perché sono codificati con un margine di errore, però ovviamente troppi pacchetti persi causano un degrado nella qualità.

Key Network Characteristics

Flussi di lunga durata: uno stream video può durare minuti o addirittura ore, creando un flusso persistente e ad alto volume di dati tra client e server.

Alto volume: anche con tecniche di compressione, questi flussi consumano una quantità significativa e costante di larghezza di banda, dell'ordine di diversi Mbps fino a decine di Mbps.

Prevedibilità: nello streaming, il traffico viene generalmente richiesto in burst prevedibili, man mano che il client riempie il proprio buffer di riproduzione.

Sensibilità ai ritardi di accodamento: il principale nemico della qualità delle applicazioni multimediali non è la scarsa larghezza di banda, bensì la variabilità dei ritardi di accodamento (*jitter*) introdotti dai router della rete.

Uso prevalente di UDP (per le applicazioni in tempo reale): le applicazioni interattive preferiscono utilizzare il protocollo **UDP**, poiché evita i ritardi causati dai meccanismi di affidabilità di **TCP**, come ritrasmissioni e controllo della congestione.

QoS Requirements by Application Class

Voice	One-way latency ≤ 150 ms
	One-way peak-to-peak jitter ≤ 30 ms
	Per-hop peak-to-peak jitter ≤ 10 ms
	Packet loss ≤ 1 percent
Broadcast video	Packet loss ≤ 0.1 percent
Real-time interactive video	One-way latency ≤ 200 ms
	One-way peak-to-peak jitter ≤ 50 ms
	Per-hop peak-to-peak jitter ≤ 10 ms
	Packet loss ≤ 0.1 percent
Multimedia conferencing	One-way latency ≤ 200 ms
	Packet loss ≤ 1 percent
Multimedia streaming	One-way latency ≤ 400 ms
	Packet loss ≤ 1 percent

Digitizing Audio And Video

Per inviare un video nella rete devo prima fare 3 passaggi:

1. **Campionamento (Sampling):** il segnale analogico continuo viene campionato a intervalli regolari, ottenendo una sequenza di campioni discreti. La **frequenza di campionamento** (numero di campioni al secondo, misurata in Hz) determina la fedeltà della rappresentazione del segnale. Ad esempio, l'audio di qualità CD utilizza una frequenza di 44,1 kHz.
2. **Quantizzazione (Quantization):** l'ampiezza di ciascun campione viene approssimata al valore più vicino di una scala discreta di valori numerici. Il numero di bit per campione (**bit depth**) determina la gamma dinamica, cioè il numero di livelli di intensità che possono essere rappresentati. Ad esempio, l'audio dei CD utilizza una profondità di 16 bit.
3. **Codifica (Encoding o Compressione):** la sequenza di campioni ottenuta occupa molto spazio. Per questo viene elaborata da un encoder (o codec), un algoritmo che comprime i dati eliminando le ridondanze spaziali e temporali, riducendo notevolmente la dimensione finale del file senza compromettere eccessivamente la qualità percepita.
 - **Compressione intra-frame (spaziale):** comprime ogni singolo fotogramma eliminando le ridondanze presenti al suo interno, in modo simile a quanto avviene con un'immagine **JPEG**. I fotogrammi ottenuti sono chiamati **I-frame** (*Intra-coded frames*).
 - **Compressione inter-frame (temporale):** riduce ulteriormente la quantità di dati inviando, anziché un nuovo fotogramma completo, **solo le differenze rispetto ai fotogrammi precedenti o successivi**. Questa tecnica genera i **P-frame** (*Predicted frames*) e i **B-frame** (*Bi-directionally predicted frames*) ed è quella che consente il maggiore risparmio di banda.

Tipi Di Traffico

TIPO TRAFFICO	CARATTERISTICHE	ESEMPI
Elastico	Può adattarsi a variazioni di ritardo e throughput. Progettato per il TCP/IP tradizionale.	FTP, Email, Web (HTTP), Telnet, SNMP.
Inelastico	Non si adatta facilmente ai cambiamenti di performance. Diventa inutilizzabile se i vincoli temporali sono violati.	VoIP, Video Conferencing, Streaming Live, Cloud Gaming.

Esempio Real Time Traffic

Le applicazioni tradizionali (come trasferimento file, email e navigazione web) vengono valutate principalmente in base a **throughput e correttezza** dei dati: se un dato arriva in ritardo ma è completo e corretto, l'applicazione può comunque funzionare.

Le applicazioni **multimediali real-time** invece hanno anche un vincolo temporale: un dato ricevuto troppo tardi può diventare inutile, anche se non è stato perso.

Esistono due principali modelli di consegna temporale:

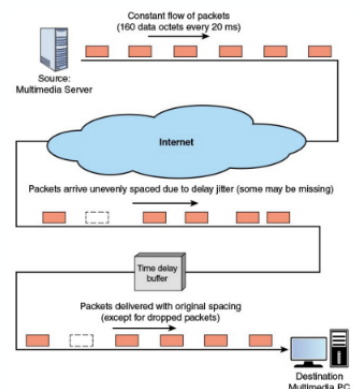
- **Constant-rate delivery:** i dati devono arrivare a una velocità circa costante, uguale a quella con cui vengono generati, per permettere una riproduzione fluida.
- **Deadline-based delivery:** ogni pacchetto ha una scadenza temporale (*deadline*); se arriva dopo il momento in cui deve essere riprodotto, può essere scartato perché non più utile.

Un'applicazione audio genera dati a **64 kbps** e invia pacchetti ogni **20 ms**. Nel mittente i pacchetti vengono prodotti a intervalli regolari, ma durante il viaggio nella rete possono subire **ritardi variabili (jitter)** a causa delle code nei router.

Per compensare questo problema il ricevitore utilizza un **playout buffer**:

- memorizza temporaneamente i pacchetti ricevuti;
- ritarda leggermente la riproduzione;
- rilascia i dati a intervalli regolari verso l'applicazione audio.

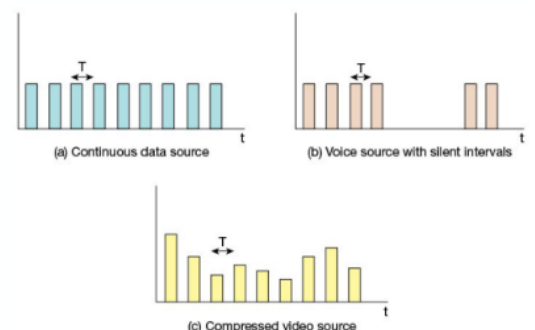
Il buffer permette di assorbire piccole variazioni di ritardo e rendere il flusso uniforme.



Come Sono Generati i Pacchetti?

Le applicazioni multimediali possono generare pacchetti in modi diversi:

- **Continuous data source:** genera pacchetti di dimensione fissa a intervalli regolari. Tipico di flussi audio o video con bitrate costante.
- **On/off source:** alterna periodi di trasmissione e periodi di inattività. Un esempio è la **voce durante una chiamata**, dove vengono inviati dati solo quando una persona parla.
- **Variable packet size source:** genera pacchetti a intervalli regolari ma con dimensione variabile. È comune nei video compressi, dove diversi frame (**I**, **P**, **B-frame**) possono avere dimensioni diverse.



Questi modelli sono importanti perché la rete deve gestire non solo la quantità media di traffico, ma anche **burst improvvisi, pause e variazioni del bitrate**.

Tipi Di Servizi Multimedia

3 grandi categorie a seconda dei limiti tempistici:

1. **Streaming di Media Archiviati:** Video pre-registrati (Netflix, YouTube). Richiedono throughput medio sufficiente a precedere il buffer di riproduzione; permettono interazioni come il riavvolgimento, pausa.
2. **Streaming di Media Live:** Eventi in diretta. La latenza dall'evento all'utente è critica (da pochi secondi a un minuto).
3. **Media Interattivi Real-time:** Comunicazione bidirezionale (VoIP). La latenza end-to-end è il fattore più critico.

Quality of Service (QoS) and Quality of Experience (QoE)

Per garantire prestazioni adeguate al traffico inelastico, i router devono implementare meccanismi di gestione differenziata:

Funzionalità dei Router

- **Controllo di Ammissione / Prenotazione Risorse:** La rete può rifiutare un nuovo flusso se non ci sono risorse sufficienti per garantirne la qualità senza degradare i flussi esistenti.
- **Classificazione e Marcatura:** I pacchetti vengono classificati (es. "voce real-time", "high-throughput video") e marcati nel campo **DSCP (Differentiated Services Code Point)** dell'header IP.
- **Accodamento e Scheduling:**
 - **Priority Queuing (PQ):** Serve sempre prima la coda a priorità più alta (ottimo per la voce, ma rischia di affamare il traffico a bassa priorità).
 - **Weighted Fair Queuing (WFQ):** Garantisce a ogni classe una percentuale specifica della banda del link in base al proprio peso.

Architetture di Rete per la QoS

QoS (Quality of Service) riguarda la rete e misura parametri oggettivi come **banda disponibile, perdita di pacchetti, latenza e jitter**.

- **Integrated Services (IntServ):** QoS per ogni singolo flusso
- **Differentiated Services (DiffServ):** QoS per categorie diverse di traffico.

Architetture di Rete per la QoE

QoE (Quality of Experience) riguarda l'esperienza dell'utente e valuta la qualità percepita, ad esempio fluidità del video, qualità dell'immagine.

- **MOS (Mean Opinion Score):** media di punteggi da 1 (Bad) a 5 (Excellent).

IntServ

Garantisce QoS per ogni singolo flusso utilizzando il protocollo **RSVP** (**ResourceReserVation Protocol**) per prenotare risorse lungo il percorso.

CLASSI

Prevede 3 classi di servizio:

- **Guaranteed Service** - forti vincoli per avere alta garanzia.
- **Controlled Load** - garantisce una qualità della rete simile a quando non è congestionata.
- **Best Effort** - niente garanzie.

REQUISITI

L'implementazione dell'architettura IntServ presenta diversi requisiti:

- Configurare tutti i **router** per supportare RSVP.
- Utilizzare un **connection admission control** per accettare o rifiutare nuove connessioni in base alle risorse disponibili.
- Mantenere nei router lo **stato di ogni flusso RSVP** e gestire la priorità dei pacchetti in base alle risorse riservate.
- Aggiornare periodicamente le prenotazioni tramite messaggi di **refresh**.
- Garantire che i flussi seguano un percorso definito; se il percorso cambia, devono essere effettuate nuove prenotazioni delle risorse.

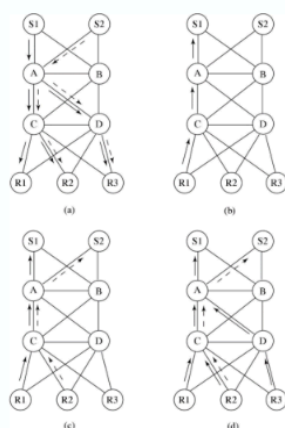
RSVP

In **IntServ (Integrated Services)**, i **Flow Specs** definiscono le risorse necessarie per un determinato flusso, mentre **RSVP** è il protocollo utilizzato per comunicare queste richieste alla rete e riservare le risorse.

RSVP è un protocollo di segnalazione per la prenotazione delle risorse nella rete, pensato soprattutto per applicazioni multimediali (anche multicast).

Caratteristiche principali:

- **Receiver-initiated:** la richiesta di prenotazione (*Resv*) viene avviata dal ricevitore e viaggia verso il mittente, seguendo il percorso della connessione.
- **Soft state:** le prenotazioni non sono permanenti; il ricevitore deve inviare periodicamente messaggi di aggiornamento (*refresh*), altrimenti la prenotazione scade.



- S1 and S2 send a *Path* message to the receivers for informing them about the route and the characteristics of the flow without reserving any resource
- R1 sends a *Resv* to S1 for reserving the resources as defined in the *Path* message
- R2 sends a *Resv* to S2 for reserving the resources
 - From A to C two separate channels should be reserved since R1 and R2 request different services
- R2 and R3 send a *Resv* to S1 for reserving the resources
 - The nodes should reconfigure by reserving separate channels or merging equivalent requests

DiffServ

Approccio scalabile basato su classi. Gestisce aggregati di traffico invece di singoli flussi, fornendo QoS specifica a diverse categorie di traffico.

In **DiffServ (Differentiated Services)**, i router applicano dei **Per-Hop Behaviors (PHB)**, cioè regole che determinano come trattare e inoltrare i pacchetti in base alla loro **classe di traffico**.

PRIORITA'

A ogni tipo di traffico può essere assegnata una diversa **priorità**, ad esempio:

- **Traffico Internet generico**: bassa priorità.
- **Streaming multimediale**: priorità media.
- **Videoconferenze e applicazioni interattive**: alta priorità.

COME MARCARE I PACCHETTI

In una rete **DiffServ**, il primo passo è **marcare i pacchetti** assegnando un valore nel campo **DS (Differentiated Services)**:

- in **IPv4** corrisponde al campo **ToS (Type of Service)**;
- in **IPv6** al campo **Traffic Class**.

La marcatura viene solitamente effettuata dal primo router della rete oppure direttamente dall'host mittente. Il router può decidere la classe del pacchetto in base a diversi parametri, come:

- indirizzo sorgente/destinazione;
- numero di porta;
- tipo di traffico.

LE CLASSI DI PHB

Le principali classi di servizio sono:

- **Default PHB**: servizio best-effort, senza garanzie particolari.
- **Expedited Forwarding (EF)**: per traffico con **basso ritardo e poche perdite**, come voce e videoconferenze.
- **Assured Forwarding (AF)**: garantisce una certa qualità di consegna in base alle condizioni stabilite.
- **Class Selector PHB**: mantiene compatibilità con vecchi sistemi.

All'interno di AF il traffico può essere ulteriormente diviso in livelli, ad esempio:

- **Gold**: 50% della banda disponibile.
- **Silver**: 30%.
- **Bronze**: 20%.

Il DiffServ è più semplice e scalabile rispetto al IntServ. I due non sono isolati ma possono lavorare insieme per garantire un servizio completo QoS.

DiffServ vs IntServ

Le tecniche **IntServ** e **DiffServ** sono state implementate in molti router moderni, ma il loro utilizzo su larga scala in Internet è limitato a causa della loro **complessità** e della difficoltà di garantire una QoS end-to-end in una rete molto grande ed eterogenea.

Per questo motivo, la maggior parte delle applicazioni multimediali assume ancora un modello **best-effort**, in cui la rete non offre garanzie precise. La qualità viene quindi migliorata tramite meccanismi a livello applicativo, come **buffering (RTP)**, **feedback (RTCP)**, **adattamento del bitrate (DASH)** e **CDN**.

Approach	Granularity	Guarantee	Mechanism	Complexity	Deployment
Best-effort	All traffic treated equally	None	No Network support	Low	Everywhere
DiffServ	Traffic class	Soft	Packet, marking, scheduling	Medium	Some
IntServ	Per-connection flow	Hard	Packet, marking, call admission	High	Little

QoS vs QoE

Una buona **QoS non garantisce sempre una buona QoE**: anche con una rete veloce e senza perdite, un servizio può avere una qualità scarsa se, ad esempio, il server invia un video a bassa qualità. L'obiettivo finale delle applicazioni multimediali è quindi **massimizzare la QoE dell'utente**, usando la QoS come strumento per raggiungerla.

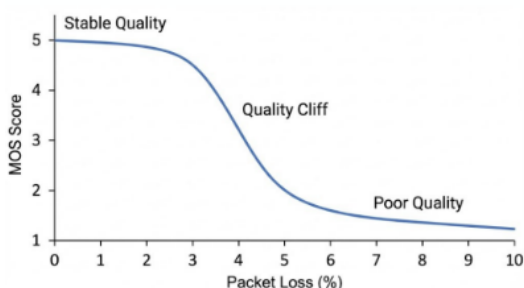
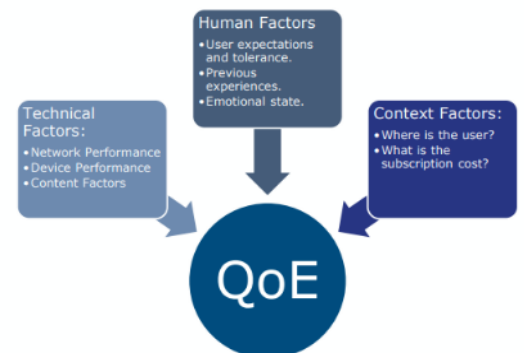
QoE

La **QoE (Quality of Experience)** misura la qualità percepita dall'utente.

Essa è soggettiva, si basa infatti su 3 fattori:

- **Fattori Tecnici**
- **Fattori Umani**
- **Fattori Contestuali**

La QoE viene spesso valutata tramite il **MOS (Mean Opinion Score)**, ottenuto facendo assegnare a un gruppo di utenti un punteggio da **1 (Bad)** a **5 (Excellent)** e calcolandone la media.



La relazione tra **QoS** e **QoE** è **non lineare**: piccoli peggioramenti della QoS possono avere un forte impatto sulla qualità percepita una volta superate determinate soglie.

Si tende a lavorare nella zona della **Stable Quality**.

Poiché i test MOS sono costosi, si utilizzano **modelli oggettivi** per stimare la QoE:

- **PSNR (Peak Signal-to-Noise Ratio)**: basato sulla differenza matematica tra immagini;
- **SSIM (Structural Similarity Index)**: che valuta la somiglianza strutturale;
- **VMAF (Video Multi-method Assessment Fusion)**: sviluppato da Netflix, che usa il machine learning ed è il più accurato nel prevedere la qualità percepita.

Per le applicazioni **VoIP**, si utilizza invece il modello **ITU-T E-Model**, che calcola un **R-Factor** (0–100) e lo converte in un punteggio **MOS**, considerando fattori come rumore, ritardo, eco e perdita di pacchetti.

$$R = Ro - Is - Id - Ie + A$$

- *Ro*: base signal-to-noise ratio
- *Is*: anomalie di segnale
- *Id*: anomalie di delay and echo
- *Ie*: anomalie di package loss
- *A*: advantage factor

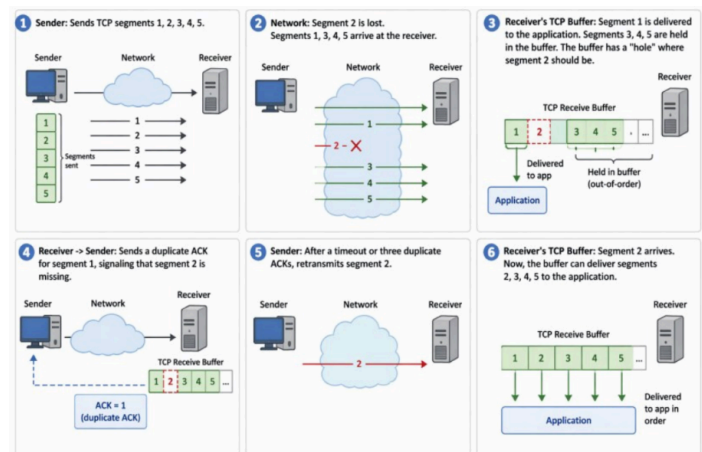
Protocolli per Real-Time e Stored Streaming

Perché TCP non è adatto al multimedia real-time?

TCP garantisce affidabilità, ma introduce ritardi che penalizzano le applicazioni in tempo reale:

- **Head-of-Line Blocking**: se un pacchetto va perso, tutti i successivi devono aspettare la ritrasmissione.
- **Controllo della congestione**: riduce il video bitrate quando rileva perdite.
- **Ritrasmissioni**: un pacchetto arrivato troppo tardi è inutile per una chiamata o videoconferenza.

Per questo motivo le applicazioni real-time preferiscono **UDP**.



UDP al posto di TCP

UDP è un protocollo leggero che non effettua ritrasmissioni né garantisce l'ordine dei pacchetti, riducendo così la latenza.

- **No HoL Blocking**: ogni datagramma è indipendente ed è l'applicazione a decidere se aspettare o no datagrammi mancanti.
- **No ritrasmissioni**: l'applicazione riceve i datagrammi che arrivano.
- **Controllo a livello dell'applicazione**

RTP al posto di UDP

Poiché UDP non fornisce informazioni sul flusso multimediale, viene utilizzato insieme a **RTP (Real-time Transport Protocol)**, che aggiunge sopra al UDP informazioni di tipo:

- numeri di sequenza (rilevare perdite e riordinare pacchetti);
- timestamp (ricostruire il corretto timing del media e compensare il jitter);
- identificazione del codec utilizzato.

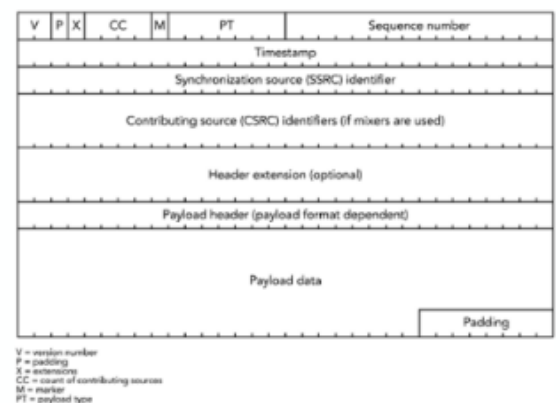
RTP non garantisce affidabilità né QoS e non presenta nessun setup di connessione o meccanismi di teardown.

RTP Header

Ogni pacchetto RTP contiene un **header** con informazioni necessarie per la corretta trasmissione del flusso multimediale.

I campi più importanti sono:

- **Payload Type (PT):** indica il **codec** utilizzato (es. H.264, Opus).
- **Sequence Number:** numero progressivo che permette di rilevare **perdite di pacchetti** e riordinarli se arrivano fuori sequenza.
- **Timestamp:** indica l'istante di campionamento dei dati ed è fondamentale per **sincronizzare la riproduzione** e compensare il **jitter**.
- **SSRC (Synchronization Source):** identifica in modo univoco la sorgente del flusso multimediale.



Sono inoltre presenti altri campi di controllo (Version, Padding, Extension, Marker e CSRC Count) utilizzati per la gestione del protocollo.

Playout Buffer Nel RTP

Per compensare il **jitter**, il ricevitore utilizza un **playout buffer**, che memorizza temporaneamente i pacchetti e li riproduce a intervalli regolari.

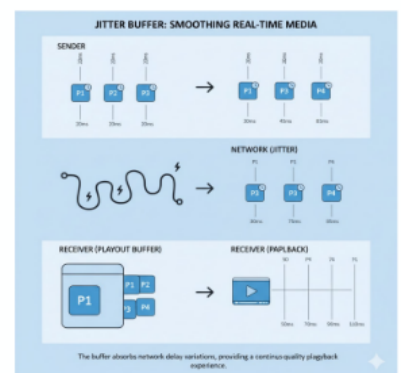
Esistono tre tipi di buffer:

- **Buffer grande:** meno interruzioni dovute al jitter ma maggiore latenza;
- **Buffer piccolo:** minore latenza ma maggiore probabilità di scartare pacchetti arrivati in ritardo.
- **Buffer adattivi:** utilizzato dai sistemi moderni, fanno resize a seconda della rete.

Jitter (variable packet arrival time) is the enemy of smooth playback. The receiver uses a **playout buffer (or jitter buffer)** to compensate for it.

1. **Sender:** Sends packets P1, P2, P3, P4 at a constant rate (e.g., every 20ms).
2. **Network:** The network introduces jitter. Packets arrive at the receiver at variable intervals (e.g., P1 at t=30ms, P2 at t=45ms, P3 at t=70ms, P4 at t=85ms).
3. **Receiver (Playout Buffer):** The packets are immediately placed into the buffer upon arrival.
4. **Receiver (Playback):** The receiver's media player pulls packets out of the buffer at a **perfectly constant rate** (every 20ms) and plays them.

Result: The buffer absorbs the network jitter, providing a smooth, continuous stream to the user.



RTCP

Il RTP gestisce solo un trasporto di media unidirezionale. Come faccio a sincronizzare molteplici stream? Come faccio a ottenere un feedback della mia stream?

RTCP (RTP Control Protocol) è un out-of-band control protocol che accompagna RTP e fornisce informazioni di controllo. Le sue funzioni principali sono:

- inviare **feedback sulla QoS** (perdite e jitter);
- da informazioni sul timing per **sincronizzare diversi flussi di media** come audio e video per il lip-sync;
- identificare i partecipanti alla sessione tramite il Canonical Name CNAME (**session control**).

I ricevitori inviano **Receiver Report (RR)** con statistiche di rete, mentre i mittenti inviano **Sender Report (SR)** con informazioni sul flusso. Il mittente può adattare il bitrate o il codec in base ai report ricevuti.

RECEIVER REPORT - RR	SENDER REPORT - SR
Campi principali: SSRC: identifica il mittente dello stream. NTP Timestamp: timestamp assoluto di 64 bit basato sul tempo reale (wall-clock). È il campo fondamentale per sincronizzare stream diversi dello stesso mittente (es. audio e video). RTP Timestamp: timestamp RTP corrispondente allo stesso istante dell'NTP Timestamp. Sender Packet Count: numero totale di pacchetti inviati. Sender Byte Count: numero totale di byte inviati.	Campi principali: SSRC: identifica il receiver che invia il report. Report Block: contiene informazioni su ogni sorgente: SSRC_n: identifica la sorgente a cui si riferisce il report. Fraction Lost: percentuale di pacchetti persi dall'ultimo RR. Cumulative Packets Lost: numero totale di pacchetti persi. Extended Highest Sequence Number Received: ultimo numero di sequenza ricevuto (usato per monitorare perdite e ordine dei pacchetti). Interarrival Jitter: misura statistica della variazione del tempo di arrivo dei pacchetti, cioè il network jitter.

SIP

Il RTP e RTCP sono perfetti per gestire il trasporto dei dati nei flussi di stream ma non gestiscono tutto ciò che riguarda il setup di una connessione e teardown.

SIP (Session Initiation Protocol) è un protocollo text-based, simile all'HTTP, di segnalazione che gestisce:

- locazione dell'utente, tramite IP;
- instaurazione della chiamata;
- negoziazione dei codec (tramite **SDP**);
- modifica e chiusura della sessione.

Terminata la fase di segnalazione, audio e video vengono trasportati tramite **RTP/RTCP**.

HAS e MPEG-DASH

La consumazione di video online si è evoluta dal **download completo**, che richiede lunghi tempi di attesa prima della riproduzione, al **progressive download HTTP**, che permette di iniziare la visione del video mentre il file viene scaricato.

Tuttavia, questo metodo non è adattivo: in caso di peggioramento della rete possono verificarsi blocchi e non è possibile accedere facilmente a parti del video non ancora scaricate.

Per questo negli anni 2000 nasce un nuovo paradigma, attualmente usato per lo streaming moderno, il **HTTP Adaptive Streaming (HAS)**:

- **Segmentation**: il video è diviso in piccoli **segmenti**;
- **Multiple Encodings**: ogni segmento è disponibile in varie qualità;
- **Client-Side Intelligence**: il client (browser/app video player) sceglie dinamicamente quale qualità scaricare in base alle condizioni della rete.

Lo standard internazionale più diffuso è **MPEG-DASH (Dynamic Adaptive Streaming over HTTP)**. Il DASH è **indipendente dal codec**, definisce esclusivamente il formato del manifest file e la struttura dei segmenti. Ciò significa che può essere utilizzato per distribuire video compressi con qualsiasi codec.

Architettura MPEG-DASH

Prima dello streaming:

- Il video viene codificato in più **Representation** (diverse risoluzioni e bitrate);
- Ogni versione viene suddivisa in **segmenti** uguali di tempo.

Il server mette a disposizione un file **MPD (Media Presentation Description)** che contiene:

- qualità disponibili;
- codec, bitrate e risoluzione;
- durata e URL dei segmenti;

Algoritmo del client DASH

Il client:

1. **Scarica il file MPD** e inizializza le rappresentazioni;
2. **Stima la banda disponibile** misurando il tempo di download dei segmenti precedenti;
 - $\text{Throughput} = (\text{Size_of_last_segment_in_bits}) / (\text{Download_time_in_seconds})$
3. Monitora il **livello del playback buffer**;
 - buffer pieno: posso anche aumentare la qualità dato che col buffer posso compensare improvvisi cali di velocità di download.
 - buffer vuoto: conviene diminuire la qualità.

Se il livello del buffer è sotto un determinato target ri-stimo la banda.

4. **Sceglie la Representation** più alta compatibile con banda e buffer;
 - $\text{estimated_throughput} > \text{representation_bitrate} * \text{safety_margin}$
5. Scarica continuamente i **segmenti successivi**.

L'obiettivo è ottenere **la massima qualità possibile evitando il buffering**. È anche molto importante avere nel buffer sempre almeno qualche secondo di content video futuro (target).

DASH per streaming live

Nel live streaming l'obiettivo principale è ridurre il **ritardo end-to-end**.

Poiché i segmenti vengono prodotti in tempo reale:

- il file **MPD** viene aggiornato periodicamente;
- il client deve sincronizzare il proprio orologio e richiedere sempre il segmento più recente disponibile.

Tabella Di Protocolli A Seconda Del Tipo Di Servizio

Tabella che a seconda del servizio ci indica il protocollo più adatto da usare:

Service Type	Main QoS Priority	Typical Stack	Typical Buffering
Interactive media (VoIP, video conferencing)	Very low delay and jitter	UDP + RTP/RTCP + SIP	Very small playout buffer
Live streaming	Low end-to-end delay, continuity	HTTP adaptive streaming + CDN	Small-to-moderate buffer
Stored streaming	Sustained throughput, no stalls	HTTP adaptive streaming + CDN	Larger playback buffer

Content Delivery Networks (CDNs)

Perché le CDN: Il problema del Server di Origine Singolo

L'uso di un unico server centrale per servire milioni di utenti globali presenta tre criticità principali,:

- **Latenza:** Gli utenti lontani dal server subiscono ritardi elevati.
- **Scalabilità:** Un singolo server diventa un collo di bottiglia quando le richieste simultanee sono troppe.
- **Affidabilità:** Un guasto al server centrale interrompe il servizio per tutti gli utenti.

Le CDN risolvono questi problemi replicando i contenuti su molti **server edge** posizionati geograficamente vicino agli utenti finali.

Architettura e Meccanismi di Funzionamento

Una CDN si basa su tre componenti fondamentali:

- **Origin Server:** La fonte autorevole del contenuto.
- **Edge Servers (o Surrogate):** Nodi di cache distribuiti nella rete.
- **Request-routing system:** Il sistema che indirizza ogni richiesta dell'utente al server edge ottimale in base a vicinanza, carico o disponibilità del contenuto.

Un utente inizialmente manda la richiesta URL di contenuto all'origin server, il quale attraverso metodi di routing sceglie quale edge server servirà all'utente il contenuto.

Metodi di Request Routing

Esistono due tecniche principali per mappare un hostname (es. video.service.com) all'indirizzo IP del miglior server edge:

- **DNS-Based Routing:** Il metodo più comune. Il DNS della CDN restituisce un indirizzo IP diverso a seconda della posizione geografica del resolver DNS dell'utente.
È semplice e scalabile, ma può essere meno preciso a causa della cache DNS, che salvano indirizzi passati al loro interno, e del fatto che spesso la posizione del DNS resolver non corrisponde con quella effettiva dell'utente.
Per migliorare la precisione, si usa l'estensione **ECS (EDNS Client Subnet)** che comunica la sottorete dell'utente reale al CDN DNS resolver.
- **Anycast-Based Routing:** Diversi server in località diverse presentano lo stesso indirizzo IP. In seguito tramite il protocollo **BGP (Border Gateway Protocol)** la rete internet indirizza naturalmente i pacchetti verso il server "più vicino" in termini di hop di rete. È una soluzione estremamente veloce e resiliente, ideale per la mitigazione di attacchi DDoS.

Strategie di Caching e Gerarchia

Il contenuto può arrivare ai server edge in due modi:

- **Pull-Based (Reattivo):** In caso il contenuto non sia salvato in cache (cache miss). Il server edge preleva il contenuto dall'origine solo quando un utente lo richiede per la prima volta.
- **Push-Based (Proattivo):** Il proprietario del contenuto invia i file ai server edge prima della richiesta. Si fa ciò per contenuti molto richiesti (es. il debutto di un film di successo o un aggiornamento software).

Per evitare di sovraccaricare il server di origine, le CDN sono spesso organizzate in una **gerarchia a più livelli**.

Edge cache → **Regional/Parent cache** → **Origin server**, così le richieste vengono soddisfatte il più vicino possibile all'utente.

Le CDN possono anche accelerare contenuti dinamici tramite **route optimization** e **TCP optimization**, facendo da proxy tra utente e server originale.

La combinazione **DASH + CDN** è fondamentale nello streaming moderno: DASH divide il video in piccoli segmenti indipendenti facilmente memorizzabili nelle cache, mentre la CDN indirizza ogni richiesta del segmento verso l'edge server più efficiente.

Nel **live streaming**, la CDN riceve un unico flusso dall'evento, lo converte in segmenti DASH a più bitrate e li distribuisce agli edge server, permettendo a milioni di utenti di guardare contemporaneamente senza sovraccaricare la sorgente.

Caso di Studio: Netflix OpenConnect

Netflix rappresenta l'esempio più avanzato di evoluzione dalle CDN di terze parti a una propria infrastruttura dedicata chiamata **Open Connect**.

- **Open Connect Appliances (OCA):** Sono server specializzati e ottimizzati per l'alto volume video, installati direttamente all'interno delle reti degli ISP o nei principali punti di interscambio (Internet Exchange Points IXP).
- Questo modello permette una collaborazione diretta con gli ISP, riduce drasticamente il traffico verso l'esterno della rete dell'operatore e migliora drasticamente la **Quality of Experience (QoE)** per l'utente finale.

Numerical Examples & Future Trends

ESEMPIO 1 - CALCOLO DEL VIDEO BITRATE

Calcolo il bitrate di un video full HD dati i seguenti dati:

- **Risoluzione:** 1920 pixels (width) x 1080 pixels (height) = 2,073,600 pixels per frame.
- **Color Depth:** 24 bits per pixel (8 per colore).
- **Frame Rate:** 30 frame per secondo (fps). Standard per video content.
- **Calcolo del bitrate:**
 - **Bits totali per frame** = 2,073,600 pixels/frame * 24 bits/pixel = 49,766,400 bits/frame.
 - **Bitrate (bits per second)** = 49,766,400 bits/frame * 30 frames/sec = 1,492,992,000 bps.
- **Risultato:** ~1.49 Gbps.

Valore ingestibile per qualsiasi internet user. Devo comprimere nel codec e rimuovere due tipi di ridondanze:

- **Ridondanze spaziali:** per un singolo frame.
- **Ridondanze temporali:** per molteplici frame.

Misuriamo l'efficienza di un codec tramite il **rapporto di compressione**.

$$\text{Compression Ratio} = (\text{Uncompressed Size}) / (\text{Compressed Size})$$

Attraverso l'uso di codec come l'H.264 (con un rapporto di compressione di 300:1), tale bitrate scende a circa **5,0 Mbps**, rendendo il contenuto fruibile su normali connessioni domestiche.

$$\text{Compressed Bitrate} = 1,492,992,000 \text{ bps} / 300 = 4,976,640 \text{ bps.}$$

Result: ~5.0 Mbps.

ESEMPIO 2 - SIMULAZIONE DEL DASH CLIENT

Viene analizzato il comportamento di un utente su un treno.

In stazione ho una banda di **~12 Mbps**, in treno dentro il tunnel ho una banda di **~3 Mbps**, fuori dal tunnel ho una banda di **~15 Mbps**.

Il client adatta dinamicamente la qualità a seconda della banda: passa dal profilo **1080p (8 Mbps)** in stazione, a un profilo **480p (1,5 Mbps)** dentro un tunnel per evitare interruzioni (buffer stall), per poi tornare all'alta definizione una volta recuperata la connettività 5G all'uscita.

Distributed Machine Learning

Perché usare il Machine Learning nelle reti?

Le ML portano 4 principali vantaggi:

Complexity Gap

Con l'evoluzione verso le reti 5G/6G e l'Internet of Things, le reti diventano troppo complesse per essere gestite con algoritmi tradizionali basati su regole ("if-then-else"). Il Machine Learning permette di affrontare problemi con moltissime variabili e condizioni che cambiano continuamente.

Predizione

Il ML consente di prevedere eventi futuri per agire in anticipo:

- **Previsione del traffico** per allocare le risorse prima che la rete si congestion.
- **Previsione della Quality of Experience (QoE)** degli utenti a partire dai parametri tecnici della rete.
- **Previsione della mobilità** degli utenti per effettuare handover più efficienti tra celle e ridurre errori e continui cambi di cella ("ping-pong").

Classificazione

Il ML riconosce schemi nei dati per:

- individuare **anomalie e guasti** nella rete;
- rilevare **attacchi informatici** (es. DDoS o attacchi sconosciuti);
- classificare il **tipo di traffico** (Zoom, YouTube, videogiochi, ecc.) per applicare le corrette politiche di QoS.

Controllo e ottimizzazione

Il ML permette di gestire automaticamente la rete:

- allocando dinamicamente le risorse radio (PRB) e computazionali;
- ottimizzando gli algoritmi di streaming video (ABR) per ridurre il buffering;
- migliorando l'efficienza energetica, spegnendo componenti inutilizzati quando il traffico è basso.

In sintesi: il Machine Learning trasforma la rete da **reattiva** (risponde agli eventi) a **proattiva e autonoma** (anticipa e ottimizza le decisioni).

I Tre Muri Del Machine Learning Centralizzato

L'idea tradizionale di raccogliere tutti i dati nel cloud per addestrare un unico modello centralizzato presenta tre grandi limiti.

Data Gravity (limite di banda)

Con quantità enormi di dati (petabyte o exabyte), trasferire tutto al cloud diventa troppo costoso e lento.

In particolare:

- stazioni radio e dispositivi IoT producono continuamente dati;
- la rete finirebbe per consumare banda solo per trasportare i dati necessari all'addestramento del modello.

Latenza

Molte applicazioni richiedono decisioni quasi istantanee.

Il problema è che il Machine Learning centralizzato richiede un percorso:

Edge → Cloud → Edge

che introduce una latenza di circa **50-150 ms**.

Applicazioni come veicoli autonomi, robot chirurgici, Internet tattile, richiedono invece tempi **< 10 ms**.

Per questo motivo addestramento e inferenza devono essere eseguiti direttamente all'Edge.

Privacy

I dati raccolti ai bordi della rete sono spesso molto sensibili:

- posizione dell'utente;
- cronologia di navigazione;
- dati sanitari;
- audio e video.

Centralizzarli comporta:

- maggior rischio di attacchi informatici;
- possibili violazioni delle normative.

Le principali leggi coinvolte sono:

- **GDPR (Europa)**, che limita il trasferimento dei dati personali;
- **HIPAA (USA)**, che protegge i dati sanitari.

Questi limiti impongono il passaggio dal paradigma "Big Data" (centralizzato) a quello di **"Wide Data"** (distribuito): la strategia non è più "portare i dati al modello", ma **"portare il modello ai dati"**.

Fondamenti del Distributed Machine Learning (DML)

L'idea fondamentale del **Distributed Machine Learning (DML)** è:

Portare il modello ai dati, invece di portare i dati al modello.

- **Machine Learning centralizzato:** tutti i dati vengono inviati al cloud o a un data center, dove il modello viene addestrato.
- **Distributed Machine Learning:** i dati rimangono nei dispositivi locali; il modello viene inviato ai nodi, addestrato localmente e vengono condivisi solo i parametri aggiornati, non i dati grezzi.

Paradigma	Concetto	Operatività	Contesto d'uso
Data Parallelism	Dataset diviso in "shards", diviso tra più nodi.	Ogni nodo addestra una copia locale del modello completo.	Cluster cloud per accelerare il training.
Model Parallelism	Modello troppo grande per un solo nodo.	Layer differenti del modello sono ospitati su nodi diversi.	Training di Large Language Models (LLM) massivi.

Oltre al training, l'ecosistema Edge-AI si definisce attraverso tre domande guida che definiscono **3 pilastri delle DML**:

1. **Federated Learning:** "Dove sono i dati?" — Restano locali, si scambiano solo aggiornamenti.
2. **Transfer Learning:** "Cosa riutilizziamo?" — Si adatta conoscenza pregressa a nuovi task con pochi dati.
3. **Split Learning:** "Dove si esegue la rete neurale?" — Il modello è partizionato tra client ed edge server.

Federated Learning

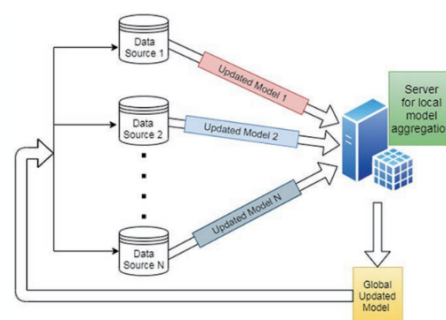
Il Federated Learning rappresenta il ponte tra AI e privacy. Il workflow operativo segue un ciclo in quattro fasi:

1. **Distribuzione:** Il server trasmette i pesi del modello globale (W_t) a un sottoinsieme di client. Si distribuisce quindi il modello globale a una serie di client.
2. **Local Training:** Ogni nodo addestra il modello sui propri dati privati (Dataset D_k) localmente per un numero di epoche E .
3. **Upload degli Aggiornamenti:** I nodi inviano al server solo i pesi aggiornati, mai i dati grezzi.
4. **Aggregazione:** Il server combina gli aggiornamenti (solitamente via *Federated Averaging*) per generare il nuovo modello globale W_{t+1} .

I 3 Pilastri del FL

- **Data Locality**
- **Decentralizzazione**
- **Network Efficiency**

La conoscenza è condivisa ma i dati sensibili rimangono protetti, salvati solo localmente.



Phase	Action	What Moves?
1. Distribution	The central server broadcasts the current model to all nodes (hospitals, phones).	Global Model
2. Local Training	Each node trains the model locally on its private dataset.	Nothing (Data stays put)
3. Update Upload	Nodes send only the updated parameters (gradients) to the server.	Local Updates
4. Aggregation	The server combines updates (e.g., Federated Averaging) to improve the model.	Improved Model

Cross-Device vs. Cross-Silo

Esistono 2 tipi di FL in base alle necessità:

Caratteristica	Cross-Device FL	Cross-Silo FL
Scala	Millioni di client (Smartphone, IoT). Dataset piccolo per client.	Pochi client (2-100 organizzazioni). Dataset grandi per client.
Affidabilità	Bassa (dropout per batteria o rete).	Alta (server dedicati 24/7).
Addestramento	avviene solo quando il dispositivo è in carica, collegato al Wi-Fi e inutilizzato.	Aggressiva e pianificata.
Esempi	Gboard (predizione testo mobile).	Ricerca medica tra ospedali o rilevamento frodi bancarie.

Problemi Del FL

Nella pratica, il Federated Learning (FL) è molto più complesso della teoria a causa di quattro problemi principali:

- **eterogeneità dei dispositivi** (hardware e connessioni diverse, “Straggler Problem”);
- **eterogeneità dei dati** (dati non distribuiti uniformemente, cioè Non-IID, “Client Drift”);
- **costi di comunicazione**;
- **sicurezza** (client malevoli o con dati rumorosi).

Questi ultimi rompono l’assunzione per cui nei modelli AI distribuiti, i dati sono **Independent and Identically Distributed (IID)**, ovvero distribuiti in modo indipendente e uniforme.

Eterogeneità Dei Dati (Non-IID): La diversità dei dati locali si manifesta attraverso quattro tipi di skew (asimmetrie):

1. **Label Skew:** ogni client vede classi differenti (es. ospedali con pazienti diversi).
2. **Feature Skew:** i dati hanno caratteristiche differenti (es. foto di città vs natura).
3. **Quantity Skew:** alcuni utenti producono molti dati, altri pochissimi.
4. **Temporal Skew:** il comportamento degli utenti cambia nel tempo.

Questi fattori causano il “**Client Drift**”, dove i modelli locali si adattano troppo ai propri dati e divergono da quello globale, rallentandone la convergenza.

Eterogeneità Dei Dispositivi: I dispositivi non hanno tutti le stesse prestazioni:

- CPU/RAM diverse;
- batterie differenti;
- connessioni Wi-Fi o 4G/5G instabili.

Questo causa il “**Straggler Problem**” ovvero quando il server aspetta tutti i client per aggiornare il modello:

- basta un dispositivo molto lento (**straggler**) per rallentare l'intero processo.

Tecniche Per Ridurre Il Traffico Dati

Anche se non vengono trasferiti i dati, i modelli moderni hanno milioni o miliardi di parametri. La quantità di dati trasmessi diventa molto elevata, esistono per questo 2 tecniche per ridurre tale traffico:

- **Model Quantization**
 - riduce la precisione dei pesi (es. da 32 bit a 8 bit);
 - diminuisce notevolmente la dimensione degli aggiornamenti.
- **Sparsification e Pruning**
 - **Sparsification:** invia solo i gradienti più importanti.
 - **Pruning:** elimina pesi o connessioni poco utili dal modello.

Algoritmi Avanzati: FedProx e SCAFFOLD

Per superare i limiti del protocollo FedAvg in scenari non ideali, ovvero quanto ho i problemi sopra elencati, sono stati creati 2 algoritmi:

- **FedProx:** Durante l'addestramento locale aggiunge un **termine prossimale (proximal term, guinzaglio)** alla funzione di costo. Questo termine impedisce al modello locale di allontanarsi troppo dal modello globale durante il training.

Vantaggi:

- maggiore stabilità;
 - migliore convergenza;
 - permette anche ai dispositivi più lenti di contribuire con aggiornamenti parziali.
- **SCAFFOLD:** Utilizza dei **Control Variates (fattori di correzione)** per stimare la direzione del drift locale. SCAFFOLD "sterza" attivamente gli aggiornamenti verso il consenso globale. Tuttavia, comporta un overhead di comunicazione superiore, poiché richiede la trasmissione sia dei pesi del modello (w) che dei control variates (c).

Vantaggi:

- convergenza più veloce;
- meno oscillazioni;
- meno round di comunicazione;
- funziona molto bene con dati fortemente Non-IID.

Guida alla Scelta Strategica:

- Scegli **FedProx** se la rete presenta molti *straggler* (dispositivi con risorse eterogenee) e cerchi una soluzione robusta e semplice.
- Scegli **SCAFFOLD** in scenari **Cross-Silo** con client fissi e affidabili, dove la priorità è ridurre drasticamente il numero di round di comunicazione, nonostante il pacchetto di dati per round sia più pesante.

Transfer Learning

Il Transfer Learning consiste nel **riutilizzare la conoscenza** appresa in un contesto (dominio/task sorgente) per migliorare l'apprendimento in un nuovo contesto correlato (dominio/task target).

- **Source domain/task:** dove il modello viene addestrato inizialmente.
- **Target domain/task:** nuovo ambiente in cui il modello deve operare.

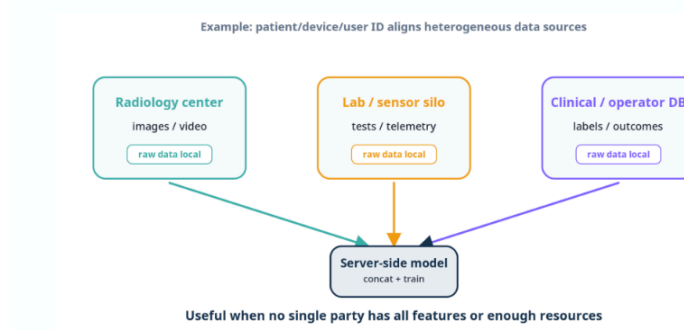
L'idea è che, se i due problemi sono simili, non è necessario addestrare un nuovo modello da zero (operazione molto costosa).

- **Utilità nelle reti wireless:** È essenziale perché gli ambienti wireless cambiano continuamente. Permette di affrontare la **scarsità di dati etichettati** in nuove celle o servizi, garantisce un **adattamento rapido** prima di raccogliere nuovi campioni e riduce sia il carico computazionale che l'overhead di comunicazione.
- **Classificazione:**
 - **Inductive TL:** Stesso dominio ma task diversi (es. passare dalla predizione del traffico alla classificazione della congestione).
 - **Transductive TL:** Stesso task ma domini diversi (es. adattare un modello di predizione da una città a un'altra).
 - **Unsupervised TL:** Assenza di etichette sia nella sorgente che nel target.
- **Strategie nel Deep Learning:** Si possono usare modelli pre-addestrati come inizializzazione, riutilizzare i primi layer (strati) dei modelli per estrarne determinate caratteristiche o effettuare il **fine-tuning** di alcuni livelli specifici.
- **Rischi:** Il pericolo principale è il **Negative Transfer**, che si verifica quando i domini sorgente e target sono troppo diversi, portando a un degradamento delle prestazioni invece che a un miglioramento.

Transfer Learning: reuse knowledge from a source task/domain



From Vertical FL to Split Learning



Split Learning

Lo Split Learning **partiziona l'esecuzione della rete neurale** tra il client e il server.

- **Architettura:** Il client conserva i dati grezzi ed esegue solo i **primi strati** della rete (tipicamente i più leggeri), mentre il server esegue i livelli rimanenti e calcola la funzione di perdita.
- **Cosa viene scambiato:** Non si trasmettono dati grezzi né pesi completi del modello, ma solo le **attivazioni intermedie** (nel forward pass fatte dal client) e i **gradienti** (nel backward pass fatte dal server) a livello del cosiddetto "split layer".
- **Work Flow:** L'addestramento è collaborativo ma i dati restano sempre sul client.
 - Il client esegue i primi layer.
 - Invia le attivazioni.
 - Il server completa la forward eseguendo i layer successivi.
 - Calcola la loss.
 - Invia i gradienti al client.
 - Client e server aggiornano ciascuno la propria parte della rete.
- **Vantaggi e Limiti:**
 - **Pro:** Riduce drasticamente il calcolo richiesto al client (ideale per dispositivi IoT limitati) e mantiene la **località dei dati**.
 - **Contro:** Richiede interazioni frequenti e costanti tra client e server; inoltre, la **latenza di rete** è critica poiché ogni fase dell'addestramento deve attraversare il collegamento.

Come Scegliere Cosa Usare?

- **Federated Learning:** quando la privacy impedisce di condividere i dati ma i client possono addestrare localmente.
- **Transfer Learning:** quando ci sono pochi dati o il problema cambia leggermente.
- **Split Learning:** quando i client hanno risorse limitate o il modello è troppo grande.

Questi approcci possono anche essere combinati (ad esempio FL + TL o FL + SL).

Paradigm	Best when...	What moves?	Main bottleneck
Federated Learning	Many clients have local data and can train the same model	Model updates	Communication rounds, non-IID data
Transfer Learning	A related model/task already exists	Knowledge, weights, features	Negative transfer, domain mismatch
Split Learning	Clients cannot train full models locally	Activations and gradients	Latency, activation leakage
Centralized ML	Data can legally and efficiently be centralized	Raw data	Privacy, bandwidth, cloud latency

Trend Futuri

AI-Driven Network Automation

Le reti moderne (5G/6G) sono troppo complesse per essere gestite manualmente.

I motivi principali sono:

- **Elevata complessità:** bisogna gestire contemporaneamente radio, slicing, edge, cloud e sicurezza.
- **Costi operativi elevati (OPEX):** configurazione e manutenzione manuali richiedono molto tempo.
- **Necessità di reazioni rapide:** congestione, guasti, mobilità e consumo energetico cambiano continuamente e richiedono decisioni in pochi millisecondi.

Per questo man mano con gli anni si è passato all'utilizzo dell'AI per automatizzare tutto.

Livelli di Autonomia

La rete evolve gradualmente dal livello **L0 (controllo umano)** al livello **L5 (autonomia completa)**. Attualmente la maggior parte delle implementazioni si trova tra **L2 e L3 (autonomia parziale)**.

Intent-Based Networking (IBN)

Introduce una gestione "**dichiarativa**" dove l'operatore specifica l'obiettivo (es. "garantisci bassa latenza per i servizi critici") e la rete decide come realizzarlo.

Le funzioni principali sono:

- **Translation:** traduce l'intento in requisiti tecnici.
- **Activation:** applica la configurazione.
- **Assurance:** verifica continuamente che l'obiettivo sia rispettato.
- **Optimization:** adatta la rete ai cambiamenti.

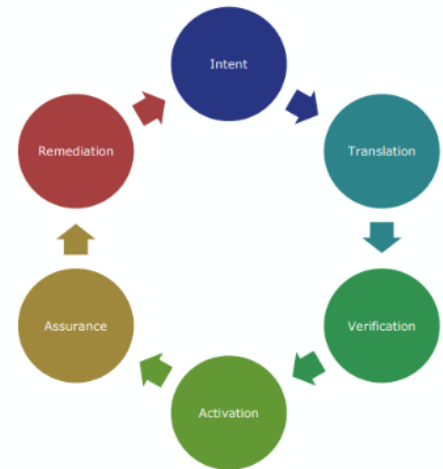
Un network intent-based esegue all'assurance facendo un ciclo di **5 fasi**:

- **Ingestion:**
 - acquisisce la richiesta dell'utente;
 - può usare NLP o LLM per interpretare il linguaggio naturale;
 - estrae requisiti come latenza, banda, priorità e utenti coinvolti.
- **Translation:** L'intento viene convertito in regole tecniche (banda, latenza, affidabilità, sicurezza). Il sistema controlla anche eventuali conflitti tra richieste diverse e suggerisce delle alternative per risolverli.
- **Verification:** Prima di modificare la rete verifica che:
 - il servizio rimanga raggiungibile;
 - siano rispettate le politiche di sicurezza;
 - non vengano creati loop o instabilità;

- **Activation:** La configurazione viene applicata tramite controller (es. SDN) e orchestratori (es. MANO). Il sistema deve presentare meccanismi di sicurezza come:
 - stop
 - rollback
 - retry

Questi servono per avere controllo sull'attivazione, la quale può portare numerosi cambiamenti alla rete.

- **Assurance:** Dopo l'attivazione la rete controlla continuamente se l'obiettivo è ancora soddisfatto. Se qualcosa cambia (guasti, traffico, mobilità):
 - rileva la deviazione (intent drift);
 - corregge automaticamente tramite:
 - rerouting;
 - scaling;
 - riconfigurazione;
 - rollback.



Zero-Touch Service Management (ZSM)

Mira ad automatizzare l'intero ciclo di vita di un servizio (configurazione, monitoraggio, ottimizzazione e terminazione). Per fare ciò bisogna quindi essere in grado di coordinare domini di diverse nature per ottenere un servizio end-to-end funzionale.

Ogni dominio (RAN, core, cloud...) è autonomo nelle decisioni locali.

Un livello superiore coordina tutti i domini per garantire gli obiettivi globali del servizio.

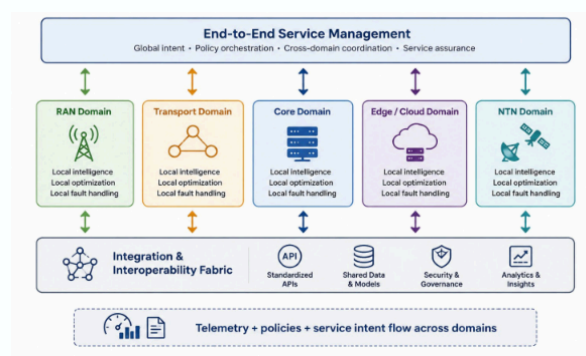
Si basa sul loop **MAPE-K** (Monitor, Analyze, Plan, Execute, Knowledge) per gestire decisioni sia a livello di singolo dominio che end-to-end.

- **Monitor:** raccoglie la telemetria della rete.
- **Analyze:** rileva anomalie e problemi.
- **Plan:** decide l'azione migliore.
- **Execute:** applica la soluzione.
- **Knowledge:** mantiene conoscenze su:
 - stati passati;
 - decisioni;
 - politiche;
 - vincoli.



Questo permette alla rete di migliorare continuamente.

ZSM Architecture: Coordinating Multiple Network Domains



Applicazioni dell'IA

Viene utilizzata per il **Self-Healing** (rilevamento proattivo dei guasti) e il **Self-Optimization** (bilanciamento tra prestazioni e consumo energetico).

L'uso di agenti di **Deep Reinforcement Learning (DRL)** permette di apprendere configurazioni ottimali.

- l'agente osserva lo stato della rete;
- sceglie un'azione;
- riceve una ricompensa;
- impara progressivamente quali decisioni sono migliori.

Provare nuove strategie direttamente sulla rete reale può essere rischioso. Per questo si usano i **Digital Twin** per addestrare l'agente in sicurezza.

Network Digital Twins (NDT)

Il **Digital Twin** è una replica virtuale dinamica della rete fisica, mantenuta costantemente sincronizzata tramite dati in tempo reale.

- **Capacità Chiave:** A differenza delle simulazioni classiche, il gemello digitale permette la **simulazione "what-if"**, consentendo di testare nuove configurazioni o aggiornamenti software senza rischiare interruzioni sulla rete reale.
- **Componenti:** Comprende la rete fisica, il modello virtuale, la connessione dati per la telemetria, i servizi del twin e uno strato di intelligenza per l'analisi dei dati.
- **Sfide:** Richiede enormi risorse computazionali, la gestione di grandi volumi di dati e garanzie sulla privacy delle informazioni degli utenti.

A simulation helps us study possible scenarios; a Digital Twin follows the real system during operation.

Classical Simulation	Digital Twin
Mostly offline	Continuously online
Uses historical, static, or synthetic data	Uses real-time data from the physical system
Mainly supports planning and validation	Supports monitoring, prediction, and optimization
Describes what could happen	Mirrors what is happening
Usually one-way: model → results	Bidirectional: physical system ↔ virtual model

Semantic Communications

Rappresenta un cambio di paradigma: l'obiettivo non è più trasmettere correttamente ogni singolo bit (era di Shannon), ma trasmettere il **significato semantico** utile per il compito del ricevitore.

Il significato semantico è la parte di un messaggio utile per il compito del destinatario. Invece di inviare ogni pixel di un video, un sistema semantico potrebbe inviare solo l'informazione

critica: *"Un veicolo si sta avvicinando a 50 km/h"*. L'informazione è considerata semantica quando:

- **Dipende dal task** del ricevitore.
- **Utilizza il contesto** e la conoscenza condivisa.
- **Privilegia i dati** che aiutano a decidere o agire, scartando quelli tecnicamente corretti ma irrilevanti.

Per capire a pieno come funziona la comunicazione semantica bisogna introdurre determinati argomenti:

- **Comunicazione Task-Oriented:** La metrica di successo diventa il completamento di un'azione (es. un robot che evita un ostacolo) invece della fedeltà del segnale.
- **Architettura Encoder e Decoder Semantici:** A differenza dei sistemi tradizionali, il sistema semantico non trasmette necessariamente dati grezzi.
 - **Encoder Semantico (Trasmettitore):** Estrae caratteristiche rilevanti (identità di oggetti, traiettorie, caratteristiche latenti).
 - **Canale:** Trasmette una rappresentazione semantica compatta.
 - **Decoder Semantico (Ricevitore):** Ricostruisce solo ciò che serve per il task. L'output potrebbe non essere identico all'originale, ma ne preserva il significato.
- **Conoscenza Condivisa:** Perché il significato sia preservato, mittente e ricevitore devono condividere un **contesto sincronizzato**. Questo richiede:
 - **Basi di conoscenza comuni:** Un background condiviso di concetti e relazioni.
 - **Ontologie comuni:** Un modo standard per descrivere cosa sono le cose e come sono connesse.
 - **Capacità di ragionamento:** Il ricevitore deve poter usare la propria conoscenza per ricostruire o interpretare messaggi compatti.
- **Ruolo dei Large Models:** Questi modelli possono aiutare nell'estrazione, compressione e ricostruzione del significato a livello di edge.
 - **Estrarre e comprimere il significato** da dati multimodali (video, audio, sensori).
 - **Comunicazione basata su prompt:** Inviare istruzioni compatte mentre il ricevitore ricostruisce le informazioni localmente.
 - **Edge Intelligence:** Utilizzare modelli più piccoli sui dispositivi per ridurre latenza e banda.